Supervised Fine-Tuning of Small Vision–Language Models for Fully Automated Chest-Radiograph Reporting: Data, Model Development, and Training Methodology

A Technical Report

Abstract

We present the design, data engineering, model development, and supervised fine-tuning (SFT) methodology behind a family of small vision–language models (VLMs) built for *fully automated* chest-radiograph (chest X-ray, CXR) reporting. Rather than treating report generation as a single free-text captioning task, we decompose the clinical act of reading a chest radiograph into a chain of explicit, machine-checkable sub-tasks: (i) *pathology classification* over a controlled vocabulary of 132 special abnormality/device tokens; (ii) *spatial grounding* via bounding-box prediction and box↔label matching; (iii) *symbolic reasoning* expressed in an Answer-Set-Programming (ASP)-style scene graph with multi-level, stepwise diagnostic analysis; (iv) *structured ontology reporting* comprising Findings, Summary, Complete Reasoning, Impression, Recommendations, and a Prominence Score; (v) *hierarchical positive-findings graphs* and finding triples; and (vi) *multi-image report comparison* for longitudinal and multi-view studies. The training corpus spans roughly 6.16 million supervised conversations drawn from public datasets (MIMIC-CXR, CheXpert), a licensed commercial source (Segmed), and a large proprietary internal cohort, with a subset of synthetic examples. We describe the full data-preparation pipeline — schema unification, per-record image/`<image>`-token validation over 100% of records, source-stratified length and prevalence analysis, empty-response detection, partial-write corruption detection, and a lazy, on-the-fly medical-image augmentation path adapted from a 2-D vision foundation pipeline. On the modeling side we compare two backbone families — InternVL3.5 (2B/8B, InternViT-300M + Qwen3 language model) and Qwen3.5 (2B/4B, a hybrid-thinking model built on GatedDeltaNet) — and document how architectural differences dictate which throughput optimizations (sequence packing, padding-free collation) are available. We detail the SFT recipe end-to-end within the ms-swift framework: full-parameter fine-tuning of the vision encoder, MLP aligner, and language model; vocabulary extension with 132 new special tokens; a non-thinking-prefix scheme with an `ignore_empty_think` loss mask; Liger-kernel fused operators; DeepSpeed ZeRO-2/ZeRO-3; and three complementary data-loading strategies (in-memory tokenization, pre-tokenized cached datasets with train-time packing, and lazy tokenization for stochastic augmentation). We report training dynamics (final train loss 0.371, eval loss 0.361, token accuracy 0.874 for the InternVL3.5-2B run over 12,571 optimizer steps) and a held-out evaluation on a curated 25,487-study high-confidence ("easy") subset drawn from a 136,639-study valid-ground-truth pool (itself distilled from 145,735 studies after quarantining 6.07% corrupt inputs), including image-integrity auditing, predicted-token distribution, per-tag and per-parent-class sensitivity/specificity/precision, and a focused nodule false-negative error analysis. On the 25k subset the model reaches a per-tag macro sensitivity of 0.874 and a 25-parent-class macro of 0.863 sensitivity / 0.863 specificity, with devices and major pathologies performing strongest. We close with a candid discussion

of failure modes and outline the path from SFT to preference optimization and reinforcement learning. This report is written as a reproducible engineering record: every dataset, hyperparameter, and analysis script referenced here is enumerated in the appendices.

Contents

1 Introduction **6**
1.1 Motivation . . . 6
1.2 What "full autoreporting" means here . . . 6
1.3 Contributions . . . 7
1.4 Document structure . . . 7

2 Preliminaries and Notation **7**

3 Background and Related Work **8**
3.1 Vision–language models and the ViT–MLP–LLM paradigm . . . 8
3.2 Automated chest-radiograph report generation . . . 8
3.3 Structured and neuro-symbolic reasoning . . . 9
3.4 Efficient supervised fine-tuning: kernels, packing, and frameworks . . . 9
3.5 Datasets in the public CXR ecosystem . . . 10

4 System Overview and Task Formulation **10**
4.1 The inference-time contract . . . 10
4.2 The response format . . . 11
4.3 Why decomposition, concretely . . . 11
4.4 Scope of this report . . . 11

5 Data: Sources and Provenance **11**
5.1 Overview . . . 11
5.2 Image sources . . . 12
5.3 Ground-truth generation and teacher models . . . 12
5.4 Two data directories and the deduplicated union . . . 13

6 Data: Schema, Unification, and the Task Families **13**
6.1 The unified record schema . . . 13
6.2 Task family 1 — Classification (controlled token set) . . . 14
6.3 Task family 2 — Structured JSON tag extraction . . . 14
6.4 Task family 3 — ASP-style symbolic reasoning . . . 14
6.5 Task family 4 — Structured ontology report . . . 15
6.6 Task families 5 & 6 — Grounding: bounding boxes and box↔label matching . . . 15
6.7 Task family 7 — Hierarchical graph and finding triples . . . 16
6.8 Task family 8 — Multi-image report comparison . . . 16
6.9 The chained pipeline and multiturn rendering . . . 16
6.10 Why structured reasoning: the case for ASP and knowledge-graph supervision . . . 16
6.10.1 The problem these representations solve . . . 16
6.10.2 Why ASP-style symbolic reasoning . . . 17
6.10.3 Why knowledge-graph (hierarchical) reasoning . . . 18
6.10.4 Why both, and why in the data rather than at inference . . . 18
6.11 Prompt inventory (top task instructions by frequency) . . . 19

6.12 Image preprocessing, photometric normalization, and coordinate frames 20

7 An End-to-End Worked Example 20
7.1 The input 20
7.2 Stage 1 — the non-thinking prefix and classification header 21
7.3 Stage 2 — the ASP symbolic scene graph and step-wise reasoning 21
7.4 Stage 3 — the fixed-schema ontology report 21
7.5 What the example demonstrates 22

8 Data: Validation, Cleaning, and Quality Auditing 22
8.1 The core structural invariant (100% coverage) 22
8.2 Empty-response auditing 23
8.3 Image-integrity auditing: partial-write corruption 23
8.4 Reproducibility of the validators 24

9 Data: Statistical Characterization 24
9.1 Length distribution by task family 24
9.2 Prompt diversity 25
9.3 Source-stratified analysis 25
9.4 Implications for training 26

10 Data: On-the-Fly Medical-Image Augmentation 26
10.1 Motivation and the core tension 26
10.2 The transform pipeline 26
10.3 The corrected zoom/crop branch 27
10.4 Integration with the Swift template: the lazy path 27

11 Data Governance, De-identification, Licensing, and Ethics 27
11.1 De-identification and PHI 27
11.2 Licensing boundaries and their training consequence 28
11.3 Intended use and the human-in-the-loop stance 28
11.4 Bias, distribution shift, and the source mix 28

12 Model Development: Backbone Selection and Architecture 28
12.1 The two backbone families 28
12.2 Why train both 29
12.3 Full fine-tuning of all three components 29

13 Model Development: Vocabulary Extension and the Controlled Token Set 29
13.1 132 new special tokens 29
13.2 Why special tokens rather than plain text 30
13.3 Embedding resize and cache coupling 30
13.4 Clinical structure of the vocabulary 30

14 Supervised Fine-Tuning Methodology 32
14.1 The common recipe 32
14.2 The non-thinking prefix and `ignore_empty_think` loss mask 32
14.2.1 Formal statement of the masked objective 33
14.2.2 Why per-component learning rates 33
14.3 Three data-loading strategies 33

14.3.1 Strategy A — in-memory tokenization (baseline) . . . 33
14.3.2 Strategy B — pre-tokenized cached dataset with train-time packing . . . 34
14.3.3 Strategy C — lazy tokenization for stochastic augmentation . . . 34
14.3.4 A decision guide . . . 34
14.4 Throughput optimizations: Liger, packing, padding-free . . . 35
14.5 Backbone-specific adaptation: the GatedDeltaNet constraint . . . 35
14.6 Global batch-size arithmetic and checkpoint resumption . . . 35

15 Training Infrastructure and Systems Engineering 35
15.1 Robust image-path resolution . . . 36
15.2 Broken-image policy (train vs. serve) . . . 36
15.3 Disk hygiene for regenerable caches . . . 36
15.4 DeepSpeed ZeRO and the memory budget . . . 36
15.5 Throughput accounting . . . 37

16 Training Dynamics and Monitoring 37
16.1 The InternVL3.5-2B run . . . 37
16.2 Reading the curves . . . 38
16.3 Monitoring stack . . . 38
16.4 Evaluation during training and checkpoint selection . . . 39
16.5 Cost and wall-clock . . . 39

17 Deployment and Serving 39
17.1 Backends . . . 39
17.2 Context-length modes . . . 39

18 Evaluation: Protocol and Data Hygiene 40
18.1 The evaluation set . . . 40
18.2 Ground truth and the join . . . 40
18.3 Two hygiene filters applied before scoring . . . 40
18.4 Metric definitions and their interpretation . . . 40
18.5 The 25k high-confidence (“easy”) subset: construction and honest reading . . . 41

19 Evaluation: Classification and Reporting Quality 42
19.1 Predicted-token distribution . . . 42
19.2 Ground-truth vocabulary and the exact-match caveat . . . 42
19.3 Consolidated per-tag metrics (25k easy subset) . . . 42
19.4 Reading the results . . . 43
19.5 Full-set context: the exact-match vs. mapped gap . . . 44
19.6 Ground-truth prevalence . . . 44
19.7 The full 25-parent-class taxonomy (25k easy subset) . . . 44
19.8 Best and worst parent classes . . . 45

20 Focused Error Analysis: Pulmonary Nodules 46
20.1 Parent nodule performance . . . 46
20.2 Why the false negatives happen (full set) . . . 46
20.3 The calcification trap (full set) . . . 46

21 Discussion 47
21.1 What worked . . . 47

21.2 What the numbers really mean 47
21.3 The precision problem 48
21.4 A taxonomy of failure modes 48
21.5 Teacher-distillation risk 48
21.6 Lessons learned 49
21.7 What we would do differently 49

22 Limitations **49**

23 Future Work **50**

24 Conclusion **50**

A Datasets and Provenance **52**

B The 132-Token Controlled Vocabulary **52**

C Full Training Hyperparameters **52**

D Cached-Dataset Export (Strategy B, Stage 1) **53**

E Inference/Serving Commands **53**

F Evaluation Data and Filters **53**

G Analysis Scripts **53**

H Reproducibility Checklist **54**

I Glossary of Terms **54**

J Selected References **55**

1 Introduction

1.1 Motivation

Chest radiography is the highest-volume diagnostic imaging examination in the world. A single large hospital network can acquire tens of thousands of chest radiographs per day, and in many parts of the world the ratio of radiologists to studies is such that a large fraction of images are never formally read, or are read only after a delay that erodes their clinical value. The economic and clinical case for *automated preliminary reporting* — a system that reads an image and produces a structured, human-readable, clinically-actionable draft report — is therefore compelling. Such a system does not replace the radiologist; it triages, pre-populates, and standardizes, so that the scarce human expert spends their attention where it matters most.

The last three years have made this ambition technically plausible. General-purpose vision–language models (VLMs) — models that fuse a vision encoder with a large language model (LLM) through a lightweight projector — have reached a level of visual grounding and language fluency where, with domain adaptation, they can describe medical images in clinically meaningful terms. The open-source ecosystem now ships strong small VLMs (2–8B parameters) that are cheap enough to fine-tune fully and to serve at scale, and mature training frameworks (ms-swift, and kernels such as Liger and Flash-Attention) that make multi-million-example supervised fine-tuning practical on a single 8-GPU node.

This report documents the end-to-end construction of such a system: a *full-autoreporting* VLM for chest radiographs. The emphasis throughout is on **data preparation and training methodology**, because in our experience these — far more than the choice of backbone — determine whether the resulting model is clinically usable. We treat the report not as an opaque string to be produced by a black box, but as the terminal node of an explicit reasoning chain that the model is trained to externalize: classify, localize, reason symbolically, and only then write the structured report. This decomposition is the organizing principle of both the data and the model.

1.2 What "full autoreporting" means here

A radiologist reading a chest film does several things almost simultaneously: they recognize abnormalities and devices, they localize them, they reason about severity and differential diagnosis in light of the clinical context, and they compose a report with a conventional structure (Findings → Impression → Recommendations). A model that only emits free-text prose short-circuits all of the intermediate structure, which makes it (a) hard to verify, (b) hard to integrate with downstream clinical software, and (c) prone to fluent-but-wrong "hallucinated" findings.

Our design instead makes each intermediate representation an *explicit, supervised output*:

- **Classification** produces a set of controlled special tokens (e.g. `<ATELECTASIS>`, `<PLEURAL_EFFUSION>`, `<ENDOTRACHEAL_TUBE>`) — a machine-readable label set that can be scored directly against structured ground truth.
- **Grounding** produces bounding boxes for findings and devices, and separately matches boxes to labels, giving spatial accountability.
- **Symbolic reasoning** produces an Answer-Set-Programming-style scene graph (`finding(...)`, `location(...)`, `severity(...)`, `characteristic(...)`) followed by step-wise diagnostic analysis — a transparent, inspectable reasoning trace.

- **Structured reporting** produces a fixed-schema ontology report: Findings, Summary, Complete Reasoning, Impression, Recommendations, and a numeric Prominence Score.
- **Hierarchical graphs and triples** encode the positive findings as a graph, giving a compact knowledge-graph view of a study.
- **Comparison** ingests two or more images and produces a structured comparison — the substrate for longitudinal follow-up and multi-view reasoning.

Crucially, these are *composed* in training data: some examples ask for one sub-task in isolation, some chain all of them into a single long reply, and some split them across multiple conversational turns. The model therefore learns both the individual skills and the full pipeline, and at inference time we can ask it for exactly as much structure as the downstream consumer needs.

1.3 Contributions

This report contributes:

1. A reproducible data-engineering methodology for assembling a ∼6.16M-conversation multi-task CXR corpus from heterogeneous sources, including a 100%-coverage structural validation invariant (image count = `<image>`-token count), source-stratified statistical characterization, and systematic data-quality auditing.
2. A task decomposition that turns report generation into a chain of machine-checkable sub-tasks, realized as a controlled vocabulary of 132 abnormality/device tokens plus ASP-style symbolic reasoning and a fixed-schema ontology report.
3. A detailed, comparative SFT methodology across two backbone families that documents how backbone architecture governs the availability of throughput optimizations, and that lays out three interoperable data-loading strategies.
4. A large-scale evaluation and error analysis over 145,735 held-out studies, including an image-integrity audit that identified and quarantined 6.07% corrupt inputs.
5. A candid account of failure modes and their causes, distinguishing genuine model limitations from artifacts of exact-match scoring against a free-form 18k-name ground-truth vocabulary.

1.4 Document structure

Sections 5–10 constitute the data half of the report and are deliberately the longest, reflecting where most of the engineering effort — and most of the eventual model quality — resides. Sections 12–17 cover model development, the SFT recipe, and the systems engineering that made multi-million-example training tractable. Sections 18–20 present the evaluation and error analysis. Sections 21–24 discuss, bound, and project the work forward. The appendices enumerate every dataset, the full 132-token vocabulary, all hyperparameters, and the analysis scripts.

2 Preliminaries and Notation

For self-containedness we fix terminology and notation used throughout; the glossary (Appendix I) gives fuller definitions.

The model. A VLM f_θ maps an instruction q and a set of images $\{I_1, \ldots, I_n\}$ to a text response y. Internally, each image is encoded by a ViT into visual tokens, projected by the

aligner into the LLM embedding space, spliced into the token stream at the positions of the `<image>` placeholders in q, and decoded autoregressively: $y \sim p_\theta(\cdot \mid q, I_{1:n})$.

The label alphabet. $\mathcal{V}$ denotes the controlled 132-token finding vocabulary (Appendix B). The classification output of a study is a subset $S \subseteq \mathcal{V}$ — the tokens the model emits in its header. A finding token $\langle X \rangle$ is *predicted-positive* for a study iff $\langle X \rangle \in S$.

Ground truth. A study's ground truth is a set G of free-form finding names (with `presence=true`). A mapping ϕ : (GT names) $\to \mathcal{V} \cup \{\varnothing\}$ associates GT names with controlled tokens; a token $\langle X \rangle$ is *GT-positive* iff some $g \in G$ has $\phi(g) = \langle X \rangle$. Parent classes group tokens into clinical super-categories (Section 13.4).

Metrics. For a fixed token/parent, over the evaluation population: TP (predicted-+, GT-+), FP (predicted-+, GT-−), FN (predicted-−, GT-+), TN (predicted-−, GT-−); sensitivity = TP/(TP+FN), specificity = TN/(TN+FP), precision = TP/(TP+FP). *Macro* averages take the unweighted mean over classes.

Data conventions. A *conversation* is one (images, turns) record; a *task family* is a JSONL file rendering a pool of studies into one instruction type; *provenance* is the source class (chexpert/mimic/segmed/internal/synthetic/prod) inferred from the image path.

3 Background and Related Work

3.1 Vision–language models and the ViT–MLP–LLM paradigm

Modern open VLMs overwhelmingly adopt a three-part architecture: a **vision encoder** (typically a Vision Transformer, ViT) that maps an image to a grid of visual tokens; a lightweight **projector / aligner** (usually a two-layer MLP) that maps visual-token embeddings into the language model's embedding space; and a **large language model** that consumes the projected visual tokens interleaved with text tokens and generates text autoregressively. This "ViT–MLP–LLM" recipe is the backbone of the LLaVA line of work and of the InternVL family, and it is the paradigm of both of our chosen backbones.

Two architectural details matter for medical imaging. First, **dynamic high resolution**: because diagnostic findings on a chest radiograph can be small (a sub-centimeter nodule, a thin pneumothorax line, the tip of a catheter), the vision encoder must ingest the image at a resolution high enough to preserve those features. InternVL's dynamic-high-resolution tiling — splitting a large image into multiple fixed-size tiles, each encoded independently — retains fine detail while keeping per-tile compute bounded. Second, **visual-token compression**: InternVL3.5 encodes each tile to on the order of 1,024 visual tokens and then compresses them (via a pixel-shuffle operation) to 256 tokens before the LLM, trading a controlled amount of spatial resolution for a 4× reduction in sequence length. Our training controls the visual-token budget explicitly through the `IMAGE_MAX_TOKEN_NUM` / `INPUT_SIZE` knobs (Section 14).

3.2 Automated chest-radiograph report generation

Automated CXR interpretation and report generation is an active field, and three generations of approaches can be distinguished. Early work used CNN encoders with RNN/transformer decoders trained end-to-end on MIMIC-CXR to produce free-text findings. A second generation built *foundation models* specifically for chest imaging: **CheXagent** (Stanford AIMI) couples a clinical LLM for parsing reports, a CXR vision encoder, and a bridging network, and is evaluated on the multi-task CheXbench suite. A third generation adds **grounding and structure**:

MAIRA-2 (a 7B radiology VLM trained on MIMIC-CXR, PadChest, and a US multi-site mix) performs *grounded* report generation — producing not only clinically accurate text but also the spatial locations of the findings it describes — and is evaluated with a sentence-level correctness/completeness framework (RadFact) that uses an LLM as judge. A parallel line of "agentic" systems (MedRAX, CXR-Agent) orchestrates multiple tools and reasoning stages with uncertainty-aware reporting.

Our work sits in the third generation but differs in emphasis. Where MAIRA-2 grounds *phrases* in a free-text report, we make the *entire intermediate representation* explicit and supervised — a controlled classification vocabulary, an ASP-style symbolic scene graph, and a fixed-schema ontology report — so that the report is the endpoint of an inspectable chain rather than a single generative leap. This is a deliberate bet: structure buys verifiability and downstream integrability at the cost of requiring far more elaborate training data, which is precisely why the data-engineering sections of this report are so extensive. Table 1 situates our system among representative CXR reporting models along the axes that matter for our design.

Table 1: Representative chest-radiograph reporting systems and where this work differs. "Controlled vocab." = a closed label set for the scored output; "Symbolic reasoning" = an explicit machine-parseable reasoning trace.

System	Backbone size	Grounding	Controlled vocab.	Symbolic reasoning
CheXagent	∼8B	no	partial (CheXbench tasks)	no
MAIRA-2	∼7B	yes (phrase boxes)	no (free text)	no
MedRAX / CXR-Agent	Agentic (tool LLM)	via tools	no	partial (tool traces)
This work	**2–8B**	**yes (boxes + match)**	**yes (132 tokens)**	**yes (ASP scene graph)**

3.3 Structured and neuro-symbolic reasoning

Free-text chain-of-thought is difficult to verify. Answer Set Programming (ASP) is a declarative logic-programming formalism in which a problem is specified as a set of logical facts and rules and solved by finding *stable models* (answer sets). We do not run an ASP solver at inference time; instead we borrow ASP's *representational discipline*. Each study's reasoning trace begins with a symbolic scene graph written in ASP-like syntax — `finding(cardiac_silhouette).`, `location(cardiac_silhouette, mediastinum).`, `severity(cardiac_silhouette, normal).` — followed by multi-level, step-wise diagnostic analysis in natural language. The value of this format is that it forces the model to *name* each finding, its location, its severity, and its characteristics as discrete, parseable predicates before it commits to prose. This is a lightweight form of neuro-symbolic modeling: the neural model emits symbolic structure that is both a scaffold for its own reasoning and an artifact that downstream software (or a human reviewer) can inspect.

3.4 Efficient supervised fine-tuning: kernels, packing, and frameworks

Full-parameter SFT of a VLM on millions of long, multimodal examples is compute- and memory-bound, and three now-standard techniques make it tractable on a single node.

Fused kernels (Liger). Liger Kernel is a collection of Triton kernels for LLM training that fuse

micro-pipeline patterns — RMSNorm, RoPE, SwiGLU, and especially a fused linear + cross-entropy that avoids materializing the full logits tensor — into register-resident operations that minimize high-bandwidth-memory traffic. In published benchmarks Liger increases multi-GPU training throughput by roughly 20% and cuts memory use by up to 60% relative to stock HuggingFace implementations, and it composes cleanly with Flash-Attention, FSDP, and DeepSpeed. We enable Liger (`-use_liger_kernel true`) in every training run; the memory headroom it frees is what lets us push per-device batch sizes and the 28k-token context window.

Sequence packing and padding-free collation. In a naïvely batched dataset, short sequences are padded to the length of the longest sequence in the batch, and the padding tokens waste compute. *Packing* concatenates multiple variable-length examples into one long sequence up to the context window, storing sequence boundaries so that attention can be restricted within each original example. With a Flash-Attention backend, the boundaries are passed as cumulative sequence lengths (`cu_seqlens`), which lets the kernel compute attention per-segment without a quadratic mask and without cross-contamination between packed examples. Packing raises the fraction of *meaningful* tokens per batch dramatically — reported utilization improvements from ∼10% to ∼98% in the extreme — for a 1.1–2× wall-clock speedup and ∼30% memory reduction, with no loss in quality. *Padding-free* is the collation-time cousin: it removes padding at the collator without pre-encoding the whole dataset.

Training framework (ms-swift). We build on ms-swift (the ModelScope Swift toolkit), which provides a unified `swift sft` / `swift export` / `swift infer` / `swift deploy` CLI over a very large model zoo, with first-class support for multimodal templates, full and parameter-efficient tuning, DeepSpeed, Liger, packing/padding-free, cached (pre-tokenized) datasets, and multiple inference backends (vLLM, SGLang, LMDeploy). ms-swift's *template* abstraction — which knows how to interleave `<image>` placeholders, apply the chat format, and mask the loss over designated spans — is the layer into which we inject our medical-image augmentation and path-resolution logic (Section 15).

3.5 Datasets in the public CXR ecosystem

Our corpus draws on the two canonical public chest-radiograph datasets and augments them with licensed and internal data. **MIMIC-CXR** is a large, de-identified dataset of chest radiographs with free-text reports from the Beth Israel Deaconess Medical Center. **CheXpert** is a large labeled CXR dataset from Stanford with uncertainty-aware labels for 14 observations. To these we add **Segmed**, a commercial provider of de-identified, consented multi-site medical imaging data (which contributes the plurality of our multi-source examples), a large **internal** proprietary cohort, and a small quantity of **synthetic** examples.

4 System Overview and Task Formulation

4.1 The inference-time contract

At inference, the system is a single VLM served behind an OpenAI-compatible endpoint (Section 17). It accepts one or more chest-radiograph images and a natural-language instruction, and returns text. The *instruction* selects which of the trained sub-tasks the model performs. Because the training data pairs each sub-task with a stable instruction template (Section 6), the downstream caller has a menu of well-defined behaviors — from a bare classification token set (cheap, fast, directly scoreable) to the full ontology report. The individual pieces of the ontology report are *also* independently addressable, because the training corpus contains those

single-turn variants.

4.2 The response format

Every trained response uses a consistent internal structure that mirrors the reasoning chain. A response to the full pipeline begins with an (empty) `<think>` block — the *non-thinking* marker required by the hybrid-thinking Qwen3.5 backbone (Section 14) — followed by the predicted classification tokens emitted between `</think>` and a delimited `<|ASP>|` ... `<|/ASP>|` reasoning section, and finally the structured ontology report. The classification tokens up front are not decoration: they are the machine-readable label set that our evaluation scores directly (Section 18), and they act as a *plan* that conditions the subsequent reasoning and report.

4.3 Why decomposition, concretely

Consider a study with a subtle right-apical pneumothorax and an endotracheal tube. A free-text-only model must get everything right in one generative pass, and if it omits the pneumothorax there is no intermediate signal that anything was missed. Under our decomposition, the failure is *localized and observable*: either `<PNEUMOTHORAX>` is absent from the classification header (a classification error we can measure against structured GT), or it is present in the header but dropped from the report (a report-consistency error), or the bounding-box task fails to localize it (a grounding error). Each of these is separately trainable and separately measurable. The evaluation in Sections 18–20 is only possible *because* of this decomposition.

4.4 Scope of this report

This report covers the supervised fine-tuning stage. It documents multiple training runs — InternVL3.5-2B and 8B, and Qwen3.5-2B and 4B — that share a data corpus and a recipe but differ in backbone and in which throughput optimizations are enabled. The flagship analyzed model is a Qwen3.5-2B checkpoint (`checkpoint-5191`) evaluated on 145,735 studies, complemented by an InternVL3.5-2B run whose full training curves we present. Preference optimization (DPO/MPO) and reinforcement learning (GRPO) are out of scope and are discussed only as future work (Section 23).

5 Data: Sources and Provenance

5.1 Overview

The training corpus is a **deduplicated union of nine large JSONL files** totaling **6,160,387 supervised conversations**. Each file is a *task family* (Section 6): the same underlying pool of chest radiographs is rendered into different instruction/response pairs. Because one study can appear in several task families, the 6.16M *conversations* correspond to a substantially smaller number of *unique images* — the multiplicity is intentional and is what teaches the model the full menu of behaviors. Table 2 lists the nine files, their row counts, and their roles.

Table 2: The nine task-family files comprising the training corpus.

File (task family)	Rows	Turns	Role
`bbox_comparison_550k`	562,619	2	Multi-image finding comparison (2–3 images)
`bbox_label_matching_617k`	617,858	2	Box↔label matching (coords in prompt)
`cls_token_mimic_chex_segmed`	637,941	2	Classification (controlled token set)
`combined_cls_asp_report`	455,382	2	Full pipeline in a single reply
`multiturn_cls_asp_report`	455,382	6	Same pipeline split across 3 turns
`prod_cls_token`	852,340	2	Production classification (deploy dist.)
`hierarchical_graph`	1,451,706	2	Hierarchical graph + finding triples
`report_comparison_deepseek_490k`	490,796	2	Report comparison (DeepSeek-generated)
`full_tag_extraction`	636,363	2	Structured JSON tag extraction
Total	**6,160,387**		

The `combined_cls_asp_report` and `multiturn_cls_asp_report` files have *identical* row counts (455,382) because they are the same source samples rendered two ways: the *combined* form puts all three tasks (classify → ASP-reason → ontology report) in one assistant turn (2 messages), while the *multiturn* form splits the same content across three user/assistant exchanges (6 messages). Training on both teaches the model to produce the pipeline either as one monolithic response or interactively.

5.2 Image sources

Each conversation's images resolve to one of six provenance classes, inferred deterministically from the image path:

- **`chexpert`** — Stanford CheXpert studies.
- **`mimic`** — MIMIC-CXR studies (Beth Israel).
- **`segmed`** — licensed, consented, de-identified multi-site data from Segmed; the largest contributor to the multi-source families (∼45–55% of rows).
- **`internal`** — a large proprietary internal cohort; dominant in the grounding/comparison families (85–88% of the bbox families).
- **`synthetic`** — a small quantity of synthetic images (∼10–14% of the bbox families), used to enrich the bounding-box coordinate distribution.
- **`prod`** — the production distribution (852,340 classification rows), representing the true deployment image mix.

The source mix is **stable across the mimic/chex/segmed task families** (∼24% chexpert, ∼31% mimic, ∼45% segmed) because those families are drawn from the same study pool and differ only in how each study is rendered into a task. Section 9 shows systematic differences in output length and finding prevalence by source (chexpert studies carry more findings per study and produce the longest outputs; segmed carries the most "no-abnormality" studies).

5.3 Ground-truth generation and teacher models

The supervised targets were generated by a mixture of human-derived labels and strong teacher LLMs applied to source reports and structured findings. **Classification tokens** derive from

structured finding labels mapped onto the controlled 132-token vocabulary (Section 13). **The ASP reasoning and ontology reports** were synthesized by prompting strong LLMs (Gemini-class models for the segmed/chexpert/mimic report families — cf. the `_gemini_response` file naming — and DeepSeek for the report-comparison family — cf. `_deepseek_490k`) with the study's findings and the target schema, so that each target is a well-structured instance of the desired output. **Bounding boxes** derive from the internal detection pipeline (and synthetic augmentation) at a known target resolution (the `1000res` / `target_res` naming records the coordinate reference frame; Section 6.6).

This teacher-distillation approach is standard for bootstrapping structured behaviors that no public dataset supplies directly (there is no public dataset of ASP scene graphs for chest radiographs). Its principal risk — that the student inherits the teacher's errors and stylistic tics — is real and is discussed in Section 21. The mitigations applied at data-prep time were: (a) using the *original* structured findings, not the teacher's free text, as the source of the classification labels we actually score; (b) validating every record structurally (Section 8); and (c) auditing for and quarantining the teacher's failure modes (empty responses, API-error strings).

5.4 Two data directories and the deduplicated union

The corpus was assembled from two staging directories that overlap. The reconciliation rules were:

1. The dataset common to both directories — `bbox_comparison_550k` (562,619 rows, byte-identical and clean in both) — is included **once**.
2. Where a file exists in a broken and a corrected version, the **corrected** version is used. Specifically, `08_June_2026_final_report_comparison_deepseek_490k` was found to be **100% broken** and was replaced by `23_June_2026_corrected_final_report_comparison_deepseek_490k`.
3. `1_Aug_final_data_for_pretraining_cleaned` was **intentionally excluded** from the final production runs.

The care taken here is not incidental: an earlier training run crashed because a regenerable dataset cache filled the shared root disk, and a report-comparison file that looked valid was in fact entirely broken. Both incidents motivated the validation regime of Section 8 and the disk-hygiene practices of Section 15.

6 Data: Schema, Unification, and the Task Families

6.1 The unified record schema

Records come in two structural variants; the training template handles both identically.

Variant A — single image. A top-level `image` string and a `messages` list:

```
{
  "image": "/localstorage/cxr_data/.../view1_frontal.png",
  "messages": [
    {"from": "human", "value": "Classify the abnormalities from the given chest xray.\n<image
    >"},
    {"from": "gpt",   "value": "<SURGICAL_HARDWARE> <CONSOLIDATION> ..."}
  ]
}
```

Variant B — multiple images. A top-level `images` list and a `conversations` list:

```
{
  "images": ["/path/img1.png", "/path/img2.png"],
  "conversations": [
    {"from": "human", "value": "... <image> ... <image> ..."},
    {"from": "gpt",   "value": "..."}
  ]
}
```

The invariants across both variants: the **image field** is `image` (single) or `images` (list); the **conversation field** is `messages` or `conversations`; each **turn** is `{"from": "human"|"gpt", "value": "..."}`; and the **`<image>` placeholder** appears inline in the human turn(s), exactly once per image, marking where the projected visual tokens are spliced into the token stream. The single most important structural invariant — verified over 100% of records (Section 8) — is:

> *The number of images in a record equals the number of `<image>` tokens in its conversation.*

A mismatch here is catastrophic: the template would splice visual tokens into the wrong position or drop an image entirely, silently corrupting every example that survives.

6.2 Task family 1 — Classification (controlled token set)

Instruction (fixed): "Classify the abnormalities from the given chest xray." **Response:** a whitespace-separated sequence of controlled special tokens, e.g. `<SURGICAL_HARDWARE> <CONSOLIDATION> <CARDIOMEGALY>`. This is the single most frequent prompt in the corpus: combining the plain, production, and pipeline classification families, "Classify the abnormalities..." accounts for **32.0%** of all prompts (3,401,451 occurrences). The response is short (median ∼55 characters for the plain family; ∼462 for the richer production family), and the label set is drawn from the 132-token vocabulary of Appendix B. A non-trivial fraction of classification targets are legitimately **empty** — a genuinely normal study has no abnormality tokens. In the plain family, 1,744 of 637,941 rows (0.27%) have an empty response; these concentrate in the segmed source (1,007), consistent with segmed carrying more no-abnormality studies (Section 8).

6.3 Task family 2 — Structured JSON tag extraction

Instruction (fixed): "Classify the abnormalities in the given chest x-ray in JSON format with presence, size, location and very short characterization." **Response:** a JSON object enumerating each finding with `presence`, `size`, `location`, and a short `characterization` (median ∼530 characters, up to ∼5,500). It is the format most directly comparable to the structured ground truth used in evaluation, whose GT ontology is itself a nested `finding` → `{presence, size, location, characteristics}` structure (Section 18).

6.4 Task family 3 — ASP-style symbolic reasoning

Instruction: e.g. "Generate Answer Set Programming (ASP)-based reasoning for the provided medical image. Include multi-level diagnostic analysis and structured, step-wise clinical reasoning." **Response:** a delimited `<|ASP>|` section that opens with a first-person framing, then a **symbolic scene graph in ASP syntax**, then multi-level step-wise analysis. An abridged real target:

```
<|ASP|>I am directly viewing the chest X-ray of this 52-year-old male, evaluating for any
    cardiopulmonary abnormalities...

STEP 1: SYMBOLIC SCENE GRAPH REPRESENTATION (ASP Format)
% ---- Findings ----
finding(cardiac_silhouette).
location(cardiac_silhouette, mediastinum).
severity(cardiac_silhouette, normal).
characteristic(cardiac_silhouette, normal_size_and_configuration).

finding(lung_parenchyma).
location(lung_parenchyma, bilateral).
severity(lung_parenchyma, normal).
characteristic(lung_parenchyma, well_aerated).
characteristic(lung_parenchyma, clear).
characteristic(lung_parenchyma, no_focal_opacities).
```

These are the **longest responses in the corpus**: the ASP-reasoning content dominates the two `cls_asp_report` families, whose median response length is ∼21,500 characters (∼5.4k tokens) and reaches ∼46,000 characters (∼11.5k tokens). Consequently this family dominates training *compute*: a single packed 28k-token sequence may hold only one or two of these examples, whereas it holds dozens of short classification examples.

6.5 Task family 4 — Structured ontology report

Instruction: "Generate a detailed ontology report of the given X-ray with Findings, Summary, Complete Reasoning, Impression, Recommendations, and Prominence Score." (plus single-section variants: "Generate the Findings…", "Generate the Complete Reasoning…", "Generate a medical report …with Findings, Impression, and Recommendations."). **Response:** a fixed-schema report with the six named sections. The **Prominence Score** is a numeric salience estimate that ranks how conspicuous/important the findings are — a triage-oriented signal. Providing both the full six-section report *and* independently-addressable single sections in the training mix is what lets the deployed model answer either "give me everything" or "just give me the Impression".

6.6 Task families 5 & 6 — Grounding: bounding boxes and box↔label matching

Two complementary grounding families teach spatial accountability. **Bounding-box generation** ("Generate the bbox of diseases and medical devices present in the image."; 16.0% of prompts, 1,700,312 occurrences) asks the model to *emit* boxes for findings and devices. The coordinate reference frame is fixed by the data preparation (the `1000res` / `target_res` naming records that boxes are expressed against a known target resolution), and the file names encode the pixel-format handling (`monochrome2` — the DICOM photometric interpretation for grayscale radiographs — so that inverted-grayscale studies are normalized). A multi-image variant ("Generate the bounding boxes for the given images. Return the results as structured JSON."; 4.7%) returns JSON boxes across two images.

Box↔label matching embeds the coordinates *in the prompt* (e.g. "…regions of interest in the chest X-ray `[[659, 212, 679, 234]]`…") and asks the model to name the finding at those coordinates. This is the one family with genuine prompt diversity: because the coordinates vary per example, **446,644 of 617,858 prompts (72.3%) are unique**. The two grounding

directions together train the model both to localize a named finding and to name a localized region.

6.7 Task family 7 — Hierarchical graph and finding triples

Instruction (fixed): "Generate a hierarchical positive findings graph and positive findings triples for the given image." (13.5% of prompts, 1,437,004 occurrences; the single largest file at 1,451,706 rows). **Response:** a hierarchical graph of the *positive* findings plus a set of finding triples — a compact knowledge-graph view (median ∼1,661 characters, up to ∼27,600). This family bridges the flat classification token set and the fully-elaborated ontology report: it captures the *structure* of a study's findings (which co-occur, how they nest under parent classes) without the full prose.

6.8 Task family 8 — Multi-image report comparison

The `bbox_comparison_550k` family (2–3 images per record; 90% two-image, 10% three-image) produces a structured comparison of findings across views/timepoints. The `report_comparison_deepseek_490` family uses a richer instruction — "Compare the radiographic findings across the images. Analyze each image individually, identify key similarities and differences, infer relative disease severity, and summarize the comparison using a structured knowledge graph. Return the final output as JSON." (4.2% of prompts) — and yields a JSON knowledge-graph comparison (median ∼10,364 characters). This is the substrate for **longitudinal reading** (today's film vs. a prior) and **multi-view reasoning** (frontal + lateral), both central to real radiology workflows and absent from single-image captioning.

6.9 The chained pipeline and multiturn rendering

The two `cls_asp_report` families realize the *full pipeline* with the chained instruction (4.24% of prompts, 450,765 occurrences): classify → ASP-reason → ontology report. In the *combined* rendering the model produces all three as one assistant turn; in the *multiturn* rendering the same content is split into three exchanges. Training on both teaches the model that these sub-tasks compose, and lets a deployed application request the whole chain at once or drive it interactively.

6.10 Why structured reasoning: the case for ASP and knowledge-graph supervision

Two of the most expensive task families to construct — the ASP symbolic reasoning (family 3) and the hierarchical positive-findings graph with triples (family 7), together with the knowledge-graph comparison output of family 8 — are also the two whose necessity is least obvious. A skeptic could reasonably ask: if the deliverable is a radiology report, why spend the teacher-distillation budget to generate 450,765 ASP-reasoning examples and 1,451,706 hierarchical-graph examples at all, rather than simply pairing images with reports? This subsection answers that question in detail, because the decision is the intellectual core of the data design and it shaped the entire corpus.

6.10.1 The problem these representations solve

A chest-radiograph model that emits only a flat token set or only free-text prose has three structural deficiencies that no amount of additional image–report pairs can fix. **(1) A flat classification set has no internal relations.** "<CARDIOMEGALY> <PLEURAL_EFFUSION> <PULMONARY_EDEMA>"

records three findings but not that they co-occur as a coherent picture of heart failure, nor which is the dominant process and which are downstream. **(2) Free-text prose has relations but they are not machine-accessible.** A report may say "findings are consistent with fluid overload," but a downstream system cannot query, verify, or aggregate that inference without re-parsing natural language — the very ambiguity we are trying to remove. **(3) Neither exposes the reasoning.** A flat label or a polished paragraph is a *conclusion* with the derivation discarded, so an error is undiagnosable: we cannot tell whether a wrong report came from a perception failure (the finding was never seen), a localization failure, or a reasoning failure (the finding was seen but misinterpreted). The ASP and knowledge-graph representations are engineered to remove exactly these three deficiencies — the first two by the knowledge graph, the third by the ASP trace — and that is why the corpus invests so heavily in them.

6.10.2 Why ASP-style symbolic reasoning

The ASP scene graph (Section 7) makes the model emit, before any prose, a set of logical predicates: `finding(`f`)`, `location(`f, ℓ`)`, `severity(`f, s`)`, `characteristic(`f, c`)`, plus a devices block, followed by step-wise diagnostic analysis. Supervising this format buys six distinct properties.

1. **Forced completeness (anti-omission).** The predicate template obliges the model to state a location, a severity, and at least one characteristic for *every* finding it asserts. Omission — the most clinically dangerous report error — becomes structurally visible: a `finding()` with no `location()` is a malformed record, not a silently incomplete sentence. Training on hundreds of thousands of fully-specified scene graphs pushes the model toward enumerating attributes it would otherwise skip in free prose.

2. **Perception/reasoning disentanglement.** The scene graph separates *what is observed* (the facts) from *what is concluded* (the step-wise analysis and, later, the Impression). This is precisely the separation that makes the failure-mode taxonomy of Section 22 possible: a false report can be traced to a wrong `finding()` fact (perception) versus a wrong inference over correct facts (reasoning).

3. **Machine-checkable consistency.** Because the predicates are parseable, they can be cross-checked automatically — against the classification header (does every header token appear as a `finding()`?), against anatomical constraints (a `location()` must be a valid region), and against mutual-exclusion rules. This is the lightweight neuro-symbolic bridge of Section 2.3: the neural model proposes symbolic facts and a symbolic layer (or an ASP solver, downstream) can validate or constrain them.

4. **Chain-of-thought that improves the answer.** The step-wise diagnostic analysis is genuine reasoning scaffolding. For a hybrid-thinking backbone, supervising an explicit derivation before the Impression conditions the final answer on an elaborated intermediate state, which empirically improves hard, multi-finding cases — the model "shows its work" and the work constrains the conclusion.

5. **A conditioning plan that curbs hallucination.** Emitting the scene graph first makes the subsequent report a *rendering* of already-committed facts rather than a fresh generative act. Fluent-but-unsupported sentences (the classic VLM hallucination) are suppressed because the prose is anchored to the enumerated predicates.

6. **Explainability for clinicians and regulators.** A transparent, inspectable reasoning trace is a requirement — not a nicety — for clinical AI. The ASP block is a human-readable

audit of why the model said what it said, which a reviewing radiologist can scan far faster than re-deriving the logic from prose.

The cost — these are the longest responses in the corpus (∼21.5k characters median, dominating training compute — Section 9) — is accepted precisely because the representational discipline is what converts a plausible captioner into an accountable reader.

6.10.3 Why knowledge-graph (hierarchical) reasoning

The hierarchical positive-findings graph and its triples (family 7), and the knowledge-graph comparison output (family 8), address the *relational* and *organizational* deficiencies that the ASP trace alone does not. Supervising graph output buys six further properties.

1. **Relations between findings become first-class.** A graph encodes co-occurrence, part–whole, and causal/associative links (e.g. `effusion` and `cardiomegaly` and `vascular_congestion` nesting under a cardiac-failure pattern). These edges are the substrate for pattern-level reasoning that a flat list cannot represent.
2. **Hierarchy mirrors the clinical ontology — and the evaluation.** Organizing findings under parent categories is the same structure as the 25-parent-class taxonomy used to score the model (Section 19.7). Training the model to *produce* the hierarchy aligns its internal organization with the clinical super-classes, which is exactly the aggregation that repairs the free-form-GT synonym fragmentation (Section 19): a graph that places `airspace_opacity`, `consolidation`, and `basilar_opacity` under one parent is doing, at generation time, the normalization we otherwise have to do at scoring time.
3. **Triples are integration- and query-ready.** `(subject, predicate, object)` triples drop directly into a graph database or a clinical knowledge base, enabling structured queries ("all studies with a new nodule adjacent to prior scarring"), RAG over prior studies, and downstream decision rules — none of which is possible over prose without a brittle NLP extraction step.
4. **The natural structure for comparison and longitudinal reading.** Comparing two studies is intrinsically a graph operation: align finding-nodes across images and label the edges with change relations (new/resolved/enlarging/stable). This is *why* the comparison family (family 8) emits a knowledge graph rather than prose — "infer relative disease severity and summarize as a structured knowledge graph" is a graph-diff task, and supervising it as such is what gives the model a principled multi-image and follow-up capability.
5. **Canonicalization of a redundant label space.** The free-form finding vocabulary is enormous and redundant (18,083 GT names; Section 19). A hierarchical graph imposes a canonical, deduplicated structure over that space, teaching the model a stable organization rather than the surface variety of the source reports.
6. **A compact bridge representation.** The graph sits between the flat classification header and the full ontology report: richer than a token list (it has relations), cheaper than full prose (∼1.7k characters median), and directly convertible into either. It is the representation a downstream application would actually store and reason over.

6.10.4 Why both, and why in the data rather than at inference

ASP and knowledge-graph supervision are complementary, not redundant, and Table 3 makes the division of labor explicit: the ASP trace gives *vertical* depth (for each finding, its full

attribute set and the reasoning that justifies it), while the knowledge graph gives *horizontal* structure (how findings relate to one another and how studies compare). A report grounded on both is grounded in both the per-finding facts and the between-finding relations — which is what a radiologist's read actually is.

Table 3: Complementary roles of the two structured-reasoning representations.

	ASP symbolic reasoning	**Knowledge / hierarchical graph**
Primary axis	vertical (per-finding depth)	horizontal (between-finding relations)
Encodes	location, severity, characteristics, derivation	co-occurrence, hierarchy, change over time
Chief benefit	completeness, verifiability, explainability	relations, integration, comparison
Downstream use	consistency checking, audit	graph DB, RAG, longitudinal diff
Mitigates	omission, hallucination, opaque errors	flat-list poverty, GT synonym fragmentation

Finally, why bake these into the *training data* rather than bolt on a post-hoc parser at inference? Because a parser applied to free text inherits every ambiguity of that text and adds its own errors, whereas a model *trained* to emit the structure produces it natively, consistently, and jointly with the report — so the structure and the prose cannot drift out of agreement. The structure is only as reliable as it is *learned*, and learning it requires exactly the large, schema-conforming ASP and graph corpora (450k and 1.45M examples) that dominate the data-preparation effort. In short: the structured-reasoning families are expensive because they are the mechanism by which the model becomes verifiable, integrable, and comparison-capable — properties that a report-only corpus, at any scale, cannot confer.

6.11 Prompt inventory (top task instructions by frequency)

Aggregated across the whole corpus, the most frequent prompts are shown in Table 4. The distribution is deliberately *classification-heavy* (families 1–4 alone are ∼54% of prompts): classification is the cheapest, most reliably-scoreable signal and the foundation on which reasoning and reporting build. The long-tail single-section prompts exist to make each report section independently addressable at inference.

Table 4: Top task instructions by share of all prompts across the 6.16M-conversation corpus.

Rank	**Share**	**Instruction**
1	32.0%	Classify the abnormalities from the given chest xray.
2	16.0%	Generate the bbox of diseases and medical devices present in the image.
3	13.5%	Generate a hierarchical positive findings graph and positive findings triples.
4	5.9%	Classify … in JSON format with presence, size, location and characterization.
5	4.7%	Generate the bounding boxes for the given images (structured JSON, 2 images).
6	4.2%	Classify … then ASP reasoning … and finally a detailed ontology report.
7	4.2%	Compare the radiographic findings across the images (structured KG, JSON).
8	1.5%	Generate the ASP reasoning and analysis of the given image.
9–13	∼1.1% ea.	Structured JSON report; Complete Reasoning; report w/ Findings, Impression, Recs; Findings; detailed ontology report.

6.12 Image preprocessing, photometric normalization, and coordinate frames

The pixel side of the pipeline required as much care as the text side, and its details are encoded (deliberately) in the dataset file names so that the preprocessing is self-documenting.

From DICOM to PNG. Source radiographs arrive as DICOM. Converting to PNG collapses the 12–16-bit DICOM dynamic range and requires a decision about the *photometric interpretation*. DICOM chest radiographs are typically `MONOCHROME2` (higher pixel value = brighter, so bone is white, air is black) but a non-trivial minority are `MONOCHROME1` (inverted). If left unnormalized, a MONOCHROME1 study would present the model with a photographic negative — lungs white, bones dark — which is catastrophic for a vision model trained mostly on the standard convention. The `monochrome2` tag in several dataset names (`prod_cls_token_monochrome2_mapped`, `bbox_..._monochrome2_segmed`) records that the photometric interpretation was normalized to MONOCHROME2 across the corpus, so every image the model sees follows the same intensity convention.

Resolution-encoding filenames. Because different findings demand different effective resolutions — a device tip needs full resolution, a global cardiothoracic-ratio judgment does not — the pipeline supports encoding a *target resize* directly in the filename, of the form `image_<W>x<H>_<original>.png`. The loader (Section 15) parses the `<W>x<H>` out of the basename, recovers the original filename to locate the file on disk, loads it, and applies a final LANCZOS resize to exactly $W \times H$. This lets the data-prep stage pin the exact resolution each example was authored for — for instance, the bounding-box families are authored against a fixed target (the `1000res` / `target_res` naming) so that box coordinates and pixels share one reference frame.

Coordinate frames for grounding. A bounding box is meaningless without the resolution it is expressed against. The grounding families fix this by construction: boxes are expressed in the coordinate frame of the target resolution recorded in the filename, and the box-generation prompts and box↔label-matching prompts both operate in that same frame. This is why the resolution-encoding convention and the grounding data were designed together — a box like `[[659, 212, 679, 234]]` is interpretable only because the image it refers to is guaranteed to be presented at the resolution the box was drawn on.

Grayscale-to-RGB and normalization. The vision encoders expect 3-channel RGB. A single-channel radiograph is expanded by channel-replication (the same gray plane in R, G, and B), after rescale/scale/clip intensity normalization (Section 10). This preserves the encoder's pretrained statistics while feeding it the grayscale content faithfully.

7 An End-to-End Worked Example

To make the abstract task decomposition concrete, this section walks a single hypothetical study through the full pipeline as the trained model would produce it, illustrating how the pieces of Sections 6 compose into one response.

7.1 The input

A frontal chest radiograph of an intubated ICU patient, with an endotracheal tube, a right internal-jugular central venous catheter, bilateral airspace opacities, and a small left pleural effusion. The caller issues the chained instruction: "Classify the abnormalities from the given chest xray, then generate an ASP-based reasoning . . . and finally a detailed ontology report . . . "

7.2 Stage 1 — the non-thinking prefix and classification header

The response opens with the empty thinking marker (loss-masked at train time, Section 14.2) and then the controlled classification tokens — the model's committed finding set, directly scoreable against structured GT:

```
<think>

</think>

<ENDOTRACHEAL_TUBE> <CENTRAL_VENOUS_CATHETER> <AIRSPACE_OPACITY> <CONSOLIDATION> <
    PLEURAL_EFFUSION>
```

7.3 Stage 2 — the ASP symbolic scene graph and step-wise reasoning

Next comes the delimited reasoning section: a first-person framing, a symbolic scene graph naming each device and finding with its location/severity/characteristics, and multi-level analysis. Abridged:

```
<|ASP|>I am directly viewing the frontal chest radiograph of this intubated patient,
    evaluating tube/line positions and the pulmonary parenchyma.

STEP 1: SYMBOLIC SCENE GRAPH REPRESENTATION (ASP Format)
% ---- Devices and Interventions ----
device(endotracheal_tube).
location(endotracheal_tube, trachea).
characteristic(endotracheal_tube, tip_5cm_above_carina).

device(central_venous_catheter).
location(central_venous_catheter, right_internal_jugular).
characteristic(central_venous_catheter, tip_in_svc).

% ---- Findings ----
finding(airspace_opacity).
location(airspace_opacity, bilateral_lower_zones).
severity(airspace_opacity, moderate).
characteristic(airspace_opacity, patchy_confluent).

finding(pleural_effusion).
location(pleural_effusion, left).
severity(pleural_effusion, small).
characteristic(pleural_effusion, blunting_costophrenic_angle).

STEP 2: MULTI-LEVEL DIAGNOSTIC ANALYSIS
- Device check: ETT tip appropriately positioned; CVC tip in SVC, no pneumothorax.
- Parenchyma: bilateral lower-zone airspace opacities favor aspiration/edema/pneumonia;
    correlate clinically.
- Pleura: small left effusion, likely reactive.
<|/ASP|>
```

7.4 Stage 3 — the fixed-schema ontology report

Finally the six-section report, terminating in a numeric Prominence Score:

```
FINDINGS: Endotracheal tube with tip ~5 cm above the carina. Right IJ central venous catheter,
     tip in the SVC. Bilateral lower-zone patchy airspace opacities. Small left pleural
    effusion with costophrenic-angle blunting. No pneumothorax.
```

```
SUMMARY: Support devices appropriately positioned; bilateral airspace disease and a small left
     effusion.
COMPLETE REASONING: The airspace pattern in an intubated patient is most consistent with
    aspiration or hydrostatic edema; superimposed infection cannot be excluded...
IMPRESSION: 1. Appropriately positioned ETT and CVC. 2. Bilateral airspace opacities, ddx
    edema vs. aspiration vs. pneumonia. 3. Small left pleural effusion.
RECOMMENDATIONS: Clinical correlation; consider follow-up radiograph after diuresis; monitor
    tube positions.
PROMINENCE SCORE: 0.72
```

7.5 What the example demonstrates

The single response contains four independently-verifiable artifacts. The classification header (Stage 1) is scored in Sections 19–20. The ASP scene graph (Stage 2) is a parseable record — one could extract every `device(...)`/`finding(...)` predicate and check it against the header for internal consistency, or against structured GT. The ontology report (Stage 3) is the human-facing deliverable, and its Findings/Impression can be cross-checked against the header (did every classified finding make it into the report?). Crucially, each stage *conditions* the next: the classification header acts as a plan that the ASP section elaborates and the report renders. This is the concrete realization of the "decomposition buys verifiability" argument of the introduction: a free-text-only model would emit only Stage 3, with no header to score and no scene graph to check.

8 Data: Validation, Cleaning, and Quality Auditing

The corpus is far too large to inspect by hand (6.16M conversations, tens of millions of image references, response strings up to 46k characters). We therefore built a small suite of streaming validators — each reads its file line-by-line so that a 10 GB JSONL never has to be resident in memory — and ran them over 100% of records. This section documents the checks, the results, and the corrective actions.

8.1 The core structural invariant (100% coverage)

The single non-negotiable check is the image/`<image>`-token count match of Section 6: every record must have exactly as many `<image>` placeholder tokens in its conversation as it has images. This was verified on *every* record (not sampled), alongside three companion checks: valid JSON parse, image field present, and conversation field present. Image-file existence was spot-checked (∼1 in 1,000 records per file). The result, summarized in Table 5, is that all nine final files pass cleanly: **0 mismatches, 0 bad-JSON, 0 missing fields across all 6,160,387 records**, and every sampled image path exists on disk.

Table 5: Structural validation, 100% of records. All nine final files pass with zero defects.

File	Rows	Mism.	Bad JSON	No img	No conv	Img miss.
bbox_comparison_550k	562,619	0	0	0	0	0
bbox_label_matching_617k	617,858	0	0	0	0	0
cls_token_mimic_chex_segmed	637,941	0	0	0	0	0
combined_cls_asp_report	455,382	0	0	0	0	0
multiturn_cls_asp_report	455,382	0	0	0	0	0
prod_cls_token	852,340	0	0	0	0	0
hierarchical_graph	1,451,706	0	0	0	0	0
report_comparison_deepseek_490k	490,796	0	0	0	0	0
full_tag_extraction	636,363	0	0	0	0	0
Total	**6,160,387**	**0**	**0**	**0**	**0**	**0**

It is worth stressing that this clean result is the *output* of the reconciliation process of Section 5, not a lucky property of the raw data. The excluded `08_June` report-comparison file was 100% broken; had it been included, every one of its 490k rows would have been a silent corruption. The validation regime is what turned "looks fine" into "provably fine."

8.2 Empty-response auditing

A separate validator flags records whose `gpt` answer is empty or blank (detected as response-length minimum of 0). Three files contain such rows (Table 6); all others have zero.

Table 6: Rows with an empty `gpt` response.

File	Empty rows	Interpretation
cls_token_mimic_chex_segmed	1,744 (0.27%)	"no-abnormality" studies with an empty label set
multiturn_cls_asp_report	1,220 (0.27%)	the *classification turn* is empty for these samples
report_comparison_deepseek_490k	310 (0.06%)	teacher produced no comparison

These are almost certainly genuine no-abnormality cases (their source distribution is segmed-heavy: 1,007 of the 1,744 empties in the classification family are segmed). Tellingly, the `combined_cls_asp_report` twin has **zero** empties on the very same 1,220 samples, because it merges the three tasks into one non-empty reply while the `multiturn` rendering leaves the classification turn empty. Two policies are defensible: (a) keep the empties, teaching the model that "normal ⇒ emit nothing" for the classification token task; or (b) drop or backfill the ∼3,270 empty rows. We flagged them explicitly so the choice is deliberate rather than accidental; the loss-masking scheme of Section 14 (`ignore_empty_think`) already prevents the empty *thinking* block from contributing spurious loss, which is a related but distinct concern.

8.3 Image-integrity auditing: partial-write corruption

The most consequential data-quality finding concerns the images themselves rather than the JSON. Scanning image files (header-only fast mode, multiprocessing) revealed a systematic **partial-write corruption** mode: a fraction of PNGs have a valid file size but are undecodable because their leading bytes are zero-filled and only the tail holds real data. A representative example is a 4.45 MB file whose first 4 MiB (4,194,304 bytes) are zeros (zero_frac = 0.943), so

PIL cannot decode it. This is the fingerprint of an interrupted write — the file was allocated and partly flushed, then the writer died.

The impact is severe if unnoticed: during inference (and, without protection, during training) an undecodable image silently falls back to a random-noise placeholder, so any prediction made "on" it is meaningless. In the 145,735-image evaluation set, **8,850 images (6.07%) were corrupt**, 0 were missing, and 136,885 were ok. We handle this at both ends of the lifecycle:

- **At training/export time** we set the `SKIP_BROKEN_IMAGES` environment flag, which makes the image loader *re-raise* on a decode failure so that the encode layer (run with `-strict false`) *drops the whole datapoint* rather than substituting a noise placeholder. Training on noise would actively teach the model to associate garbage pixels with real labels; dropping is strictly safer.
- **At inference/serving time** the same loader falls back to a random RGB image (so a batch never crashes), but we *annotate* every output row with an `image_status` field and quarantine `status != ok` rows before scoring (Section 18).

This dual policy — drop at train, annotate-and-quarantine at eval — is the single most important data-hygiene decision in the project, because 6% is more than enough corruption to move headline metrics if left in.

8.4 Reproducibility of the validators

Every check above is a small streaming script (`validate.py` for the count invariant, `check_exist.py` for image existence, `count_empty.py` for empties, `analyze.py`/`subgroup.py` for statistics, and the `image_integrity` module for corruption). They emit machine-readable JSON/CSV artifacts (`analysis_result.json`, `subgroup_result.json`, `image_integrity_full.csv`, ...) so that any downstream change to the corpus can be re-audited in one command. The scripts are enumerated in Appendix G.

9 Data: Statistical Characterization

Understanding the length and content distribution of the corpus is essential both for setting the context window and for interpreting where training compute is spent. All lengths below are in *characters*; a rough token estimate is characters ÷ 4.

9.1 Length distribution by task family

Table 7 gives, per file, the full-conversation and response-length distributions. The pattern is stark: **prompts are short and nearly constant** (most families use a single fixed instruction template), so essentially all length variation lives in the `gpt` response. The two `cls_asp_report` families are by far the longest (median response ∼21.5k characters, p99 ∼30.7k, max ∼46k), followed by `report_comparison` (∼10.4k median). The classification families are the shortest (∼55–462 median). Because attention cost scales super-linearly with sequence length, the two report families dominate training FLOPs far out of proportion to their 15% row share — a fact that directly motivates sequence packing (Section 14), which lets many short classification examples share a sequence with one long report example rather than each padding to 28k.

Table 7: Length distribution (characters). Full conversation and `gpt` response: median / p99 / max.

File	Turns	Full (med/p99/max)	Resp (med/max)	Unique prompts
`bbox_comparison_550k`	2	728 / 2,004 / 7,766	602 / 7,623	2 templates
`bbox_label_matching_617k`	2	339 / 555 / 727	86 / 408	446,644 (72.3%)
`cls_token_mimic_chex_segmed`	2	117 / 269 / 477	55 / 415	1 template
`combined_cls_asp_report`	2	21,891 / 30,780 / 46,149	21,541 / 45,799	1 template
`multiturn_cls_asp_report`	6	22,030 / 30,919 / 46,288	21,537 / 45,795	3 templates
`prod_cls_token`	2	524 / 1,142 / 1,606	462 / 1,544	1 template
`hierarchical_graph`	2	1,768 / 6,642 / 27,739	1,661 / 27,632	1 template
`report_comparison_deepseek_492k`	2	10,660 / 15,212 / 27,707	10,364 / 27,411	2 templates
`full_tag_extraction`	2	668 / 1,887 / 5,637	530 / 5,499	1 template

9.2 Prompt diversity

Seven of the nine files use a *single* fixed instruction template (three for the multiturn file), which is exactly what one expects for task-specific SFT: all the diversity is in the image and the answer. The two exceptions carry real prompt variation. `bbox_label_matching_617k` has **446,644 unique prompts (72.3%)** because the bounding-box coordinates are embedded in the prompt text; no template recurs more than a handful of times. `bbox_comparison_550k` has two templates (a 2-image variant at 90% and a 3-image variant at 10%). This near-uniformity of prompts is a feature, not a bug: it means the model learns to map *instruction* $\rightarrow$ *behavior* crisply, and it is why the deployed system responds reliably to the exact instruction strings of Section 6.11.

9.3 Source-stratified analysis

Stratifying each multi-source file by provenance (Table 8) reveals consistent clinical structure:

- **CheXpert produces the longest outputs** in every multi-source family (e.g. hierarchical_graph: chexpert median 2,379 vs. segmed 1,460; full_tag: 728 vs. 379), because chexpert studies carry more findings per study.
- **Segmed produces the shortest outputs and the most empty responses** (1,007 of the 1,744 classification empties; 675 of the 1,220 multiturn empties), consistent with more no-abnormality studies.
- **Row-share is stable** across the mimic/chex/segmed families ($\sim$24% / $\sim$31% / $\sim$45%), since they derive from the same study pool.
- In `bbox_label_matching`, unique prompts split cleanly by source (internal 398k, synthetic 42k, mimic 5.8k); synthetic prompts are the least diverse (its outputs are near-constant, median = p90 = max = 86 characters).

Table 8: Source mix (row %) across the multi-source families.

File	chexpert	mimic	segmed	internal	synth.	prod
bbox_comparison_550k	–	1.2%	–	88.2%	10.7%	–
bbox_label_matching_617k	–	1.3%	–	85.0%	13.7%	–
cls_token_mimic_chex_segmed	23.3%	30.7%	46.0%	–	–	–
combined_cls_asp_report	24.1%	31.2%	44.7%	–	–	–
multiturn_cls_asp_report	24.1%	31.2%	44.7%	–	–	–
prod_cls_token	–	–	–	–	–	100%
hierarchical_graph	15.3%	25.7%	33.1%	25.9%	–	–
report_comparison_deepseek_490k	20.2%	24.8%	55.0%	–	–	–
full_tag_extraction	23.4%	30.8%	45.9%	–	–	–

9.4 Implications for training

Three consequences follow directly from these statistics. (1) The 28k-token `max_length` was chosen so that the p99 of the longest family (∼30.7k characters ≈ 7.7k tokens of text, plus up to a few thousand visual tokens) fits with headroom; only a handful of extreme report examples are truncated. (2) Because response length is so skewed, *packing* yields a large win — without it, a batch containing one 46k-character report example would force every co-batched short example to that length. (3) The source-length correlation (chexpert long, segmed short) means naïve length-based batching would inadvertently stratify by source; we therefore shuffle globally (`-dataset_shuffle true`, `-data_seed 42`) before splitting and packing.

10 Data: On-the-Fly Medical-Image Augmentation

10.1 Motivation and the core tension

Data augmentation — applying randomized, label-preserving transforms to images each epoch — is standard in medical imaging, where it improves robustness to acquisition variation (exposure, rotation, cropping, elastic deformation of soft tissue). But there is a fundamental tension with the efficient training pipeline. The fastest data path pre-encodes every example *once* (tokenizing text and processing pixels to `pixel_values`) and freezes the result to disk (Section 14); this is what makes packing possible. A frozen `pixel_values` tensor, however, is a *fixed* image — stochastic augmentation applied to a cached tensor would be identical every epoch, defeating its purpose. Augmentation therefore requires the image to be re-loaded and re-transformed at every `__getitem__`, which is fundamentally incompatible with pre-encoded caching and packing.

10.2 The transform pipeline

We adapted the augmentation pipeline from an internal 2-D vision foundation codebase into a Swift-compatible module (`transforms_qure.py`). It provides two callables built on `albumentations`:

- `Resize_Scale` — a deterministic square resize to a fixed `im_size` (used for evaluation/inference-style preprocessing), applying rescale→scale→clip normalization and expanding the single-channel radiograph to 3-channel RGB.
- `RandAugment` — the stochastic training transform. On *every call* it builds a fresh `albumentations.ReplayCo` pipeline:
  1. With probability 0.3, one of {elastic transform, grid distortion} — soft-tissue deformation.

2. With probability 0.5, one of {a random square resize + crop, a ±15° rotation}.
3. Always, a final resize to the target `im_size`.

followed by photometric jitter (gamma, brightness, contrast) and clipping.

10.3 The corrected zoom/crop branch

The one behavioral change from the original vision-foundation pipeline is the zoom/crop branch. The original used *fixed* sizes: `Resize(1100,1100)` → `RandomCrop(960,960)`. The corrected branch randomizes the scale: it resizes to a random square $S \times S$ with $S \sim \text{Uniform}[800, 1600]$, then random-crops to $\text{round}(0.85\,S) \times \text{round}(0.85\,S)$ — i.e. a 15% height/width reduction at a randomized zoom level. This gives the model exposure to a continuum of effective magnifications rather than a single fixed one, which matters for a domain where the same finding appears at widely varying pixel scales depending on acquisition and patient size. (Mask-carrying variants of the pipeline additionally use a `-100→2.0` sentinel-swap trick so that ignore-index mask regions survive interpolation and are restored exactly afterward — relevant when the same augmentation is reused for segmentation targets.)

10.4 Integration with the Swift template: the lazy path

Because the transform must run per-sample per-epoch, augmentation is only available on the *lazy-tokenize* data path (`-lazy_tokenize true`), in which the full `_encode` pipeline (text tokenization + image processing) runs inside `__getitem__` each epoch rather than once up-front. The integration layer converts the transform's output (a CHW float tensor in $[0, 1]$) back to a PIL RGB image before handing it to the Swift processor, which expects PIL. The training script that uses this path (`qure_full_2b_lazy_augment.sh`) deliberately sets `-packing false` (Swift raises "Packing and lazy_tokenize are incompatible" because packing pre-encodes and freezes pixels) and instead relies on `-padding_free true` for efficiency, which is collation-time-only and needs no pre-encoding. This is the crux of the caching-vs-augmentation trade-off, and the reason we maintain *two* training recipes (cached+packed for maximum throughput; lazy+augmented for maximum robustness); Section 14 lays both out in full.

11 Data Governance, De-identification, Licensing, and Ethics

Because the corpus is medical, its governance is not an afterthought to the modeling but a precondition for it. This section records the provenance obligations attached to each source and the ethical posture of the system.

11.1 De-identification and PHI

Chest radiographs and their reports are protected health information (PHI). Every source in the corpus is used in de-identified form. The public datasets (MIMIC-CXR, CheXpert) are released de-identified under their respective data-use agreements; MIMIC-CXR in particular requires credentialed access and completion of human-subjects training, and prohibits attempts at re-identification. The commercial source (Segmed) supplies consented, de-identified, multi-site data under a commercial license. The internal cohort is handled under the organization's own de-identification pipeline. Two image-level PHI risks specific to radiographs are (a) burned-in annotations (patient name/date rendered into the pixels by some modalities) and (b) identifiable

implanted-device serial markings; the DICOM→PNG conversion (Section 6.12) and source de-identification are the controls against the former, and the model is never trained to read or emit such markings.

11.2 Licensing boundaries and their training consequence

The corpus mixes licenses with different redistribution terms: MIMIC-CXR and CheXpert are research-use datasets with attribution and no-redistribution clauses; Segmed data is commercially licensed; the internal cohort is proprietary. Two consequences follow for how the artifacts of this project may be shared. First, the *trained weights* encode information from all sources and therefore inherit the most restrictive applicable terms — the model cannot be released more permissively than its most-restricted training source allows. Second, the *data files* themselves (the nine JSONL task families) are not redistributable outside the licensed environment; this report therefore documents their schema and statistics rather than shipping the data. These boundaries are why the deduplication and provenance-tracking of Section 5 matter beyond data quality: knowing exactly which source each row came from is a licensing requirement, not merely good hygiene.

11.3 Intended use and the human-in-the-loop stance

The system is designed as a *preliminary/triage* reporter, not an autonomous diagnostician. Its outputs are drafts to be reviewed and adjudicated by a qualified radiologist. This stance is reflected concretely in the design: the classification header and ASP scene graph exist precisely so that a human reviewer can quickly see *what* the model claims and *why*, and the Prominence Score exists to order a review queue rather than to make a disposition. The evaluation posture of Sections 18–20 reinforces this — we report sensitivity/precision per finding so that a deploying clinician can see, finding by finding, where the model is trustworthy (devices, effusions) and where it is not yet (mediastinal/hilar, vascular), rather than a single aggregate that would obscure the safety-relevant heterogeneity.

11.4 Bias, distribution shift, and the source mix

The source mix (Section 9) is not demographically neutral: MIMIC and CheXpert reflect two specific US academic populations, Segmed spans multiple sites, and the internal/prod distributions reflect the deployment setting. Models trained on such mixtures can under-perform on populations, scanners, or disease prevalences under-represented in training. We treat this as a known, unquantified risk here (we do not present demographic-stratified metrics), and flag demographic and cross-site evaluation as required work before any clinical deployment (Section 23). The over-prediction of ambiguous findings (Section 21) is itself a form of bias — a systematic tilt toward flagging — that is safer than under-calling in a triage setting but must be characterized, not assumed benign.

12 Model Development: Backbone Selection and Architecture

12.1 The two backbone families

We developed and trained across two open backbone families, chosen for their strong general multimodal capability, small footprint (2–8B, fully fine-tunable on one node), and mature Swift support.

InternVL3.5. An InternViT vision encoder (InternViT-300M for the 2B/8B variants) + MLP projector + a Qwen3-series language model, in the ViT–MLP–LLM paradigm. The 2B variant pairs InternViT-300M with Qwen3-1.7B (∼2.3B total); the 8B variant pairs InternViT-300M with Qwen3-8B (∼8.5B total). It retains InternVL's dynamic-high-resolution tiling and pixel-shuffle visual-token compression (1,024 → 256 tokens per tile). Critically for us, InternVL3.5 uses **standard attention** in its language model, which means sequence packing and padding-free collation are cleanly supported (Section 14).

Qwen3.5. A hybrid-thinking multimodal model whose language backbone uses **GatedDeltaNet**, a linear-attention/state-space-style mechanism. Hybrid-thinking means the model can operate in an explicit "thinking" mode (emitting a `<think>...</think>` reasoning block) or a "non-thinking" mode. The GatedDeltaNet backbone is efficient at long context but, on the transformers/Swift backend at the time of our runs, **does not support packing or padding-free** the way standard-attention models do — a constraint that shaped the Qwen3.5 recipe (Section 14.5).

12.2 Why train both

The two families are complementary. InternVL3.5's standard attention makes it the throughput champion: with packing enabled, it achieves the highest effective token utilization and the cleanest long-context training. Qwen3.5's hybrid-thinking design is attractive for a reasoning-heavy task like ours (the ASP and Complete-Reasoning sections are literally chain-of-thought), and its GatedDeltaNet backbone is memory-efficient at long context. We therefore built parallel recipes: an InternVL3.5-2B/8B track (packing-optimized) and a Qwen3.5-2B/4B track (packing-free, with the non-thinking scheme of Section 14.2). The flagship evaluated checkpoint (Section 18) is Qwen3.5-2B; the fully-plotted training run (Section 16) is InternVL3.5-2B.

12.3 Full fine-tuning of all three components

In all runs we fine-tune *every* component — vision encoder, aligner, and language model — rather than freezing the vision tower. The training flags make this explicit: `-freeze_vit false`, `-freeze_aligner false`, with per-component learning rates `-vit_lr 1e-5`, `-aligner_lr 1e-5`, matching the base `-learning_rate 1e-5`. The rationale is domain shift: a general-purpose ViT trained on natural images has never seen the grayscale, high-dynamic-range, texture-dominated statistics of a chest radiograph, nor the sub-centimeter findings that matter clinically. Freezing it would bottleneck the whole model on features that were never optimized for radiography. Full fine-tuning is expensive but, for a 2B model on 6M examples, entirely feasible on 8 GPUs, and in a specialized medical domain it is (in our experience) the difference between a model that describes images and one that reads them.

13 Model Development: Vocabulary Extension and the Controlled Token Set

13.1 132 new special tokens

The controlled classification vocabulary is realized as **132 new special tokens** added to the tokenizer and the model's embedding table (plus two delimiter tokens `<|ASP>|` and `<|/ASP>|` that bracket the reasoning section, for 134 lines in the token file). Each token names one abnormality, device, or structural finding: pathologies (`<ATELECTASIS>`, `<PLEURAL_EFFUSION>`,

<CARDIOMEGALY>, <PNEUMOTHORAX>, <CONSOLIDATION>, <PULMONARY_NODULE>, ...), devices/hardware (<ENDOTRACHEAL_TUBE>, <PACEMAKER>, <PICC_LINE>, <CHEST_TUBE>, <STERNOTOMY_WIRES>, ...), and a <NORMAL> token. The full list is Appendix B.

13.2 Why special tokens rather than plain text

Emitting <ATELECTASIS> as a single atomic token, rather than the word "atelectasis" spread over several sub-word pieces, has three advantages. First, **unambiguous scoring**: the evaluation can detect a finding by exact token match, with no tokenizer-dependent string-matching fragility. Second, **a closed label space**: the model cannot "invent" a finding outside the controlled vocabulary in the classification header, which bounds the hallucination surface for the part of the output we score. Third, **efficient representation**: a finding is one token regardless of how long its name is, so the classification header stays short even for a study with many findings. The cost is that the embedding table grows by 132 rows that must be learned from scratch, which is one more reason the language model is fully (not parameter-efficiently) fine-tuned.

13.3 Embedding resize and cache coupling

Adding special tokens resizes the model's input and output embedding matrices. This is passed at train time via `-new_special_tokens <token file>`, and — importantly — the flag *must be repeated* whenever a pre-tokenized cache is consumed, because the cache stores raw token ids that only make sense against the resized vocabulary. The training scripts encode this coupling as a hard requirement (the cached-dataset recipe re-passes `-new_special_tokens` so the embedding table is resized to match the ids baked into the cache; a mismatch would map the new ids to garbage rows). This tight coupling between the token file, the cache, and the template args (system prompt, non-thinking prefix, loss scale, max length) is why the split script's header warns that the cache is "tokenizer/template-specific" and must be regenerated if any of these change.

13.4 Clinical structure of the vocabulary

The 132 tokens are not a flat list; they organize into clinically-meaningful families, and this structure is what the parent-class evaluation (Section 19.7) later aggregates over. Table 9 groups the vocabulary into its principal clinical families with representative members. The design goal was *coverage without redundancy*: enough granularity that a token carries clinical meaning (e.g. distinguishing <SOLITARY_PULMONARY_NODULE> from <DIFFUSE_PULMONARY_NODULES> from <CALCIFIED_PULMONARY_NODULE>, which differ in workup), but bounded enough that each token has adequate training support. The device/hardware families are deliberately fine-grained (13 device/line/tube tokens plus 8 hardware/surgical tokens) because device identification and position assessment is one of the highest-value, highest-reliability tasks in ICU chest radiography — and, as Section 19 shows, it is where the model performs best (0.75–0.88 sensitivity).

Table 9: Clinical families within the 132-token vocabulary (representative members; not exhaustive).

Family	**Representative tokens**
Airspace / opacity	OPACITY, AIRSPACE_OPACITY, CONSOLIDATION, INFILTRATE, PNEUMONIA, GROUND_GLASS_OPACITIES, BASILAR_OPACITY, PERIHILAR_OPACITIES, RETROCARDIAC_OPACITY, ATELECTASIS
Pleural	PLEURAL_EFFUSION, PLEURAL_THICKENING, PLEURAL_CALCIFICATION, PLEURAL_PLAQUE, PLEURAL_SCARRING, EMPYEMA, HEMOTHORAX
Air leak	PNEUMOTHORAX, HYDROPNEUMOTHORAX, PNEUMOMEDIASTINUM, PNEUMOPERITONEUM, SUBCUTANEOUS_EMPHYSEMA
Nodule / mass / malignancy	PULMONARY_NODULE, SOLITARY_PULMONARY_NODULE, DIFFUSE_PULMONARY_NODULES, CALCIFIED_PULMONARY_NODULE, CAVITARY_NODULE, GROUND_GLASS_NODULE, GRANULOMA, CALCIFIED_GRANULOMA, MASS, MEDIASTINAL_MASS, HILAR_MASS, MALIGNANCY, METASTASIS, LESION
Cardiac	CARDIOMEGALY, PERICARDIAL_EFFUSION, HEART_FAILURE
Vascular / aortic	AORTIC_ANEURYSM, AORTIC_ECTASIA, AORTIC_TORTUOSITY, ATHEROSCLEROSIS, MPA_DILATION, PULMONARY_HYPERTENSION, PULMONARY_VENOUS_HYPERTENSION
Edema / congestion	PULMONARY_EDEMA, PULMONARY_VASCULAR_CONGESTION, KERLEY_LINES
Interstitial / fibrotic	INTERSTITIAL_LUNG_DISEASE, FIBROSIS, SCARRING, PERIBRONCHIAL_THICKENING, FISSURAL_THICKENING
Obstructive / airway	COPD, EMPHYSEMA, HYPERINFLATION, BRONCHIECTASIS, BRONCHITIS, REACTIVE_AIRWAYS_DISEASE
Osseous / chest wall	RIB_FRACTURE, HEALED_RIB_FRACTURE, CLAVICLE_FRACTURE, COMPRESSION_FRACTURE, SCOLIOSIS, KYPHOSIS, OSTEOPENIA, DEGENERATIVE_CHANGES, DEMINERALIZATION
Tubes / lines / catheters	ENDOTRACHEAL_TUBE, TRACHEOSTOMY_TUBE, FEEDING_TUBE, CHEST_TUBE, PICC_LINE, CENTRAL_VENOUS_CATHETER, DIALYSIS_CATHETER, PORT_CATHETER, PULMONARY_ARTERY_CATHETER, EPIDURAL_CATHETER
Cardiac devices	PACEMAKER, CARDIAC_DEFIBRILLATOR, LVAD, LOOP_RECORDER, PROSTHETIC_VALVE
Surgical / hardware	STERNOTOMY, STERNOTOMY_WIRES, CABG, SURGICAL_HARDWARE, ORTHOPEDIC_HARDWARE, SPINAL_HARDWARE, LUNG_SURGERY, LUNG_TRANSPLANT
Diaphragm / mediastinum	ELEVATED_DIAPHRAGM, TENTED_DIAPHRAGM, DIAPHRAGMATIC_HERNIA, MEDIASTINAL_WIDENING, MEDIASTINAL_SHIFT, TRACHEAL_SHIFT, LYMPHADENOPATHY, HILAR_ENLARGEMENT
Technical / pseudolesion	ROTATION, ARTIFACT, NIPPLE_SHADOWS, LOW_LUNG_VOLUMES, NORMAL

14 Supervised Fine-Tuning Methodology

This is the heart of the training pipeline. We describe the common recipe, the three data-loading strategies, the non-thinking/loss-masking scheme, the throughput optimizations, and the backbone-specific adaptations, and we tie each choice to a concrete flag in the `swift sft` invocation. The complete flag sets are reproduced in Appendix C.

14.1 The common recipe

Across all runs the shared hyperparameters are:

- **Tuner:** `-tuner_type full` (full-parameter fine-tuning of ViT + aligner + LLM).
- **Precision:** `-torch_dtype bfloat16`, `-attn_impl flash_attention_2`.
- **Optimization:** `-learning_rate 1e-5` (and `vit_lr`/`aligner_lr` = 1e-5), `-warmup_ratio 0.05`, `-num_train_epochs 1`. One epoch over 6.16M examples is already a very large number of gradient steps; more epochs risk overfitting the teacher-distilled style.
- **Context:** `-max_length 28000`, with `-max_new_tokens` 18000–25000 (Section 9 shows this covers the p99 of even the longest family).
- **Kernels:** `-use_liger_kernel true` everywhere (Section 14.4).
- **Parallelism:** `-deepspeed zero2` (2B) or `zero3` (8B and packed InternVL), `NPROC_PER_NODE=8`.
- **Data:** `-dataset_shuffle true`, `-split_dataset_ratio 0.01` (a 1% held-out eval split), `-data_seed 42`.
- **System prompt:** `-system 'You are a helpful medical assistant.'` (fixed, so the deployed system prompt matches training).
- **Checkpointing:** `-save_steps 800`, `-eval_steps 800`, `-logging_steps 5`, `-save_total_limit` 4–5, `-create_checkpoint_symlink true`, `-report_to tensorboard wandb`.
- **Visual-token budget:** `IMAGE_MAX_TOKEN_NUM=2500` (Qwen3.5) / `INPUT_SIZE=448` (InternVL), `VIDEO_MAX_TOKEN_NUM=128`, `FPS_MAX_FRAMES=12`.

14.2 The non-thinking prefix and `ignore_empty_think` loss mask

Both backbone families are hybrid-thinking: they support an explicit `<think>...</think>` block. Our supervised targets, however, already contain the reasoning we want (in the ASP and Complete-Reasoning sections), and we do *not* want the model to emit an *additional*, redundant free-form thinking block before the structured output. We therefore train in *non-thinking* mode: `-add_non_thinking_prefix true` prepends an *empty* `<think>\n\n</think>` marker to each target, teaching the model to open with an empty thinking block and then proceed directly to the classification tokens and structured report.

The catch: an empty thinking block is still tokens, and naïvely computing loss over it would push the model to reproduce the empty-block boilerplate as if it were content, and could destabilize the loss for examples where the block is genuinely empty. The `-loss_scale ignore_empty_think` setting solves this by *masking the loss over the empty thinking span*, so the model is supervised on the real output (tokens after `</think>`) but not penalized/rewarded for the empty marker. This pairing — `add_non_thinking_prefix` + `ignore_empty_think` — is what cleanly turns a

hybrid-thinking base model into a direct-answering structured reporter without discarding its latent reasoning capacity (which we still exploit inside the ASP section).

14.2.1 Formal statement of the masked objective

Let a training example be a token sequence $x_{1:T}$ with a binary loss mask $m_{1:T} \in \{0,1\}$. Standard causal SFT minimizes the mean next-token cross-entropy over the *assistant* tokens only:

$$\mathcal{L} \;=\; -\frac{1}{\sum_t m_t} \sum_{t=1}^{T} m_t \, \log p_\theta(x_t \mid x_{<t}) \,,$$

where $m_t = 1$ iff token t belongs to a supervised assistant span (user/system prompt tokens and the interleaved visual tokens have $m_t = 0$). Our two template settings modify m precisely:

- `add_non_thinking_prefix` inserts an empty ⟨`think`⟩⟨/`think`⟩ span into the target, at positions $\mathcal{E} \subset \{1, \ldots, T\}$.
- `loss_scale=ignore_empty_think` sets $m_t = 0$ for all $t \in \mathcal{E}$, so the empty-marker tokens contribute neither gradient nor loss normalization.

Under packing (Section 14.4), K examples $x^{(1)}, \ldots, x^{(K)}$ are concatenated into one sequence of length $\sum_k T^{(k)} \le L_{\max} = 28000$, with per-example boundaries $\{(\text{start}_k, \text{end}_k)\}$ supplied to Flash-Attention as `cu_seqlens`; the attention mask is block-diagonal so that token $t \in$ example k attends only to positions within $[\text{start}_k, t]$. The masked objective is then summed over the concatenation with each example's own $m^{(k)}$, so packing changes *throughput* but not the *loss* an example contributes — a property we rely on when comparing packed and unpacked runs.

14.2.2 Why per-component learning rates

The three trainable modules have very different pretraining exposure to radiographs (the ViT: none; the aligner: none; the LLM: whatever medical text was in its corpus). We nonetheless set $\eta_{\text{vit}} = \eta_{\text{align}} = \eta_{\text{llm}} = 10^{-5}$, a single conservative rate, rather than a higher ViT rate, for two reasons. First, the 132 new embedding rows are learned from scratch and a high global rate risks destabilizing them early; the 5% warmup (∼630 steps for the InternVL-2B run) eases this. Second, in full fine-tuning a too-aggressive ViT rate can cause catastrophic forgetting of the encoder's general visual priors before the aligner has adapted, which manifests as a loss spike we did not observe at 10^{-5}. The clean single-epoch curve of Figure 1 is partly a consequence of this conservative, uniform schedule.

14.3 Three data-loading strategies

A distinctive aspect of this project is that we maintain three interoperable data paths, each with a different throughput/flexibility trade-off. They are *not* alternatives to be chosen once; they are tools matched to a purpose.

14.3.1 Strategy A — in-memory tokenization (baseline)

The default `swift sft -dataset <files>` path tokenizes and image-processes the dataset in-memory (parallelized by `-dataset_num_proc`), caching the encoded result to the HF datasets cache (`-load_from_cache_file true`). Simple and robust; used by the InternVL3.5-8B run and the Qwen3.5-4B run. Its limitation is that with a hybrid-thinking/GatedDeltaNet backbone, packing may be unavailable, so it can leave throughput on the table for very skewed length distributions.

14.3.2 Strategy B — pre-tokenized cached dataset with train-time packing

The highest-throughput path is a two-stage process. *Stage 1 (split/export):* `swift export -to_cached_dataset true` tokenizes the entire corpus *once*, processes every image to `pixel_values`, applies the template (system prompt, non-thinking prefix, loss scale, special tokens), splits into train/val, and writes the encoded rows to disk with HF `save_to_disk`. This is where `SKIP_BROKEN_IMAGES=1` + `-strict false` drop the 6%-corrupt images (Section 8.3). Note the cache is *tokenizer- and template-specific* — it is tied to the exact model, `new_special_tokens`, `add_non_thinking_prefix`, `loss_scale`, `system`, and `max_length`; those must be identical in Stage 2. *Stage 2 (train):* `swift sft -cached_dataset <dir>/train -cached_val_dataset <dir>/val -packing true` consumes the cache with *no* tokenization or image loading at train time, and applies packing over the cached rows. A subtlety recorded in the scripts: the split writes `train/` and `val/` as *separate* datasets (no top-level `dataset_dict.json`), so each must be pointed at directly rather than via the parent directory. This is the path used for the flagship Qwen3.5-2B and InternVL3.5-2B packed runs.

14.3.3 Strategy C — lazy tokenization for stochastic augmentation

When per-epoch image augmentation is desired (Section 10), we use `-lazy_tokenize true` directly on `-dataset` (no cache): the full encode pipeline runs at every `__getitem__`, so a fresh augmentation is applied each epoch. Here `-packing` *must* be false (packing and lazy-tokenize are mutually exclusive by construction, since packing pre-encodes and freezes pixels); efficiency instead comes from `-padding_free true`, which is collation-time-only.

Table 10 summarizes the trade-off.

Table 10: The three data-loading strategies and their trade-offs.

Strategy	Packing	Augment	Encode cost	Used for
A: in-memory	maybe*	no	once (cached)	8B, 4B baselines
B: cached + packed	**yes**	no	once (offline)	flagship 2B runs
C: lazy + padding-free	no	**yes**	per epoch	augmentation runs

* depends on backbone: available for InternVL (standard attention), not for Qwen3.5 (GatedDeltaNet).

14.3.4 A decision guide

The choice among the three strategies reduces to two questions. *(1) Does this run need stochastic per-epoch augmentation?* If yes, only Strategy C is possible (packing is off by construction), and efficiency comes from padding-free collation. If no, proceed to (2). *(2) Does the backbone support packing?* If yes (InternVL / standard attention), Strategy B (cached+packed) is optimal — pay the one-time offline encode cost, then train at maximum token density. If no (Qwen3.5 / GatedDeltaNet), fall back to Strategy A with a large per-device batch and optional `group_by_length`. In practice we used B for the flagship 2B runs (both backbones, with the documented Qwen3.5 fallback), A for the 8B/4B baselines, and C when we wanted to measure the augmentation effect. The three are interoperable because they share the *same* template args (special tokens, non-thinking prefix, loss scale, system, max length); only the encode-and-batch machinery differs, so a checkpoint trained under one can be evaluated or resumed under another without template drift.

14.4 Throughput optimizations: Liger, packing, padding-free

Liger kernel (`-use_liger_kernel true`) is on in every run; its fused RMSNorm/RoPE/SwiGLU and fused-linear-cross-entropy free the memory that lets us run bf16 full-tuning of a 2B model at 28k context with a non-trivial per-device batch size. **Packing** (`-packing true`) bin-packs cached rows up to `max_length` per sequence; `per_device_train_batch_size` then counts *packed sequences*, so it is small (2–8) even though each sequence carries many examples. Packing auto-enables **padding-free** (they are one knob, not two independent ones) and makes `group_by_length` irrelevant. Both require a Flash-Attention backend, which passes per-segment boundaries as `cu_seqlens` so packed examples never attend across each other. For the InternVL3.5-8B run we additionally set `-packing_num_proc 128` to parallelize the packing index build, and (in one 8B run) `-neftune_noise_alpha 5` for NEFTune embedding noise as a mild regularizer.

14.5 Backbone-specific adaptation: the GatedDeltaNet constraint

The most important backbone-specific fact is that **Qwen3.5's GatedDeltaNet backbone does not support packing/padding-free** on the transformers/Swift backend. This forces a different efficiency strategy for the Qwen3.5 track:

- In the *in-memory* Qwen3.5 runs (2B lazy-augment, 4B), `-packing false` and `-padding_free false`; efficiency comes from a *large per-device batch* (e.g. 20) × 8 GPUs × gradient accumulation, giving a global batch of 480–720, with `group_by_length` optionally reducing padding at the cost of a noisier loss curve.
- The Qwen3.5-4B run uses global batch $720 = 6 \times 8 \times 15$ (later 20×8×3) at 28k context, ZeRO-2.
- The *cached-packing* Qwen3.5-2B recipe (`qure_full_2b_cached_packing.sh`) attempts `-packing true` against the cache and carries an explicit fallback note: if training errors on packing/padding-free (the GatedDeltaNet incompatibility), set both false — the cache is still valid, only the packing speedup is lost.

By contrast, the InternVL3.5 runs (standard attention) set `-packing true -padding_free true` cleanly, and this is why the flagship *fully-plotted* training curve is the packed InternVL3.5-2B run (Section 16).

14.6 Global batch-size arithmetic and checkpoint resumption

Global batch size is per_device × data-parallel size × gradient-accumulation. Representative settings: InternVL3.5-2B packed uses $2 \times 8 \times 8 = 128$ packed sequences (each up to 28k tokens); the InternVL3.5-8B packed run uses $1 \times 8 \times 16 = 128$; the Qwen3.5-4B run targets a global batch of 720. Long runs use `-resume_from_checkpoint` (e.g. the Qwen3.5-4B run resumes from `checkpoint-9600`) so that a multi-day training job survives interruptions, with `-save_only_model false` preserving optimizer state for exact resumption.

15 Training Infrastructure and Systems Engineering

Training a VLM on 6M examples and 8 GPUs surfaces a set of unglamorous but decisive systems problems: where do millions of images live, what happens when one is unreadable, and how do regenerable caches avoid filling a shared disk. We solved these inside the Swift template layer and the training scripts.

15.1 Robust image-path resolution

The corpus was assembled over months across several storage mounts (`/localstorage`, `/fastdata3`, `/fastdata4`), and image paths baked into the JSONL do not always match the mount on the machine actually running the job. We extended Swift's image loader with a `_resolve_image_path` helper that (a) passes HTTP URLs through untouched, (b) handles a resolution-encoding filename convention — filenames of the form `image_<WxH>_<original>.png` carry a target resize embedded in the name, which is parsed out (the `WxH` drives a final LANCZOS resize to the intended resolution) and the original filename recovered — and (c) falls back across the configured storage roots (`_resolve_storage_path` over `/localstorage, /fastdata3, /fastdata4`) until the file is found. The training scripts mirror this at the shell level, rewriting `/localstorage`↔`/fastdata4` (and `/fastdata3`) for both dataset and token-file paths so the same script runs unmodified on any node.

15.2 Broken-image policy (train vs. serve)

As introduced in Section 8.3, the loader's exception handler is environment-aware. When `SKIP_BROKEN_IMAGES` is set (training/export), it *re-raises* on any decode failure so the encode layer (`-strict false`) drops the datapoint — never training on a placeholder. When the flag is unset (inference/serving), it returns a random RGB image so a batch never crashes, and the row is annotated downstream. This single conditional is the mechanism that keeps the 6% corrupt images (Section 8.3) out of the gradients while keeping the serving path crash-free.

15.3 Disk hygiene for regenerable caches

An early export run crashed when the ModelScope/HF datasets cache filled the shared root filesystem. The scripts now redirect *all* regenerable caches off root onto a large scratch mount, resolving a writable base (`/fastdata4` then `/localstorage`) and exporting `MODELSCOPE_CACHE`, `HF_DATASETS_CACHE`, `TMPDIR`, `TRITON_CACHE_DIR`, and `TORCHINDUCTOR_CACHE_DIR` beneath it. They also cap per-worker thread pools (`OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, `TOKENIZERS_PARALLELISM=false`) so that a high `-dataset_num_proc` (up to 128 during export) does not oversubscribe the box with dozens of BLAS/tokenizer threads per worker. `PYTORCH_CUDA_ALLOC_CONF=` reduces allocator fragmentation at long context.

15.4 DeepSpeed ZeRO and the memory budget

Full fine-tuning stores, per parameter, the fp32 master weight, the bf16 working weight, and the Adam first/second moments — roughly 16 bytes/parameter of optimizer+master state on top of activations. For a 2B model that is ∼32 GB of state alone, and for 8B, ∼128 GB, neither of which fits comfortably alongside 28k-token activations on a single accelerator. We therefore shard with DeepSpeed ZeRO. **ZeRO-2** partitions the optimizer states and gradients across the 8 data-parallel ranks (each rank holds 1/8 of them) while replicating parameters; this suffices for the 2B runs and keeps communication modest. **ZeRO-3** additionally partitions the *parameters* themselves, gathering each layer's shard on-the-fly during forward/backward; we use it for the 8B runs and for the packed InternVL-2B run, where the 28k-token activation footprint leaves less room for replicated parameters. `-gradient_checkpointing true` trades compute for memory by recomputing activations in the backward pass, and is essential at 28k context. The interaction with packing is favorable: a packed sequence is always near-full ($\leq L_{\max}$), so the activation memory is predictable and there are no wasted pad activations to checkpoint — one more reason packing is the preferred path where the backbone supports it.

15.5 Throughput accounting

The effective tokens processed per optimizer step is per_device $\times$ DP $\times$ grad_accum $\times \bar{L}$, where $\bar{L}$ is the mean packed-sequence length ($\approx L_{\max}$ under packing, but far smaller under padded batching of short classification examples). This is why packing's benefit is largest for our skewed length distribution (Section 9): without packing, a global batch dominated by 55-character classification targets wastes almost the entire 28k window on padding; with packing, every step processes $\sim$128 sequences $\times$ 28k $\approx$ 3.6M real tokens. The single-epoch InternVL-2B run's 12,571 steps at this density corresponds to on the order of 10^{10} supervised tokens.

16 Training Dynamics and Monitoring

16.1 The InternVL3.5-2B run

Figure 1 shows the full training and evaluation curves for the packed InternVL3.5-2B run (Strategy B): a single epoch over the corpus, **12,571 optimizer steps**, batch of 2 packed 28k sequences per device, ZeRO-3, Liger, packing + padding-free. Training loss (logged every 5 steps; 2,515 points) falls from an initial $\sim$1.20 to a final **0.371**; the held-out eval loss (logged every 800 steps; 16 points) tracks it closely to a final **0.361**. Token accuracy rises from $\sim$0.73 to a final **0.874** (train) / **0.874** (eval). The tight train/eval agreement — eval loss slightly *below* train loss throughout — indicates the model is not overfitting at one epoch; the small gap is consistent with the eval split being drawn i.i.d. from the same distribution (`-split_dataset_ratio 0.01`).

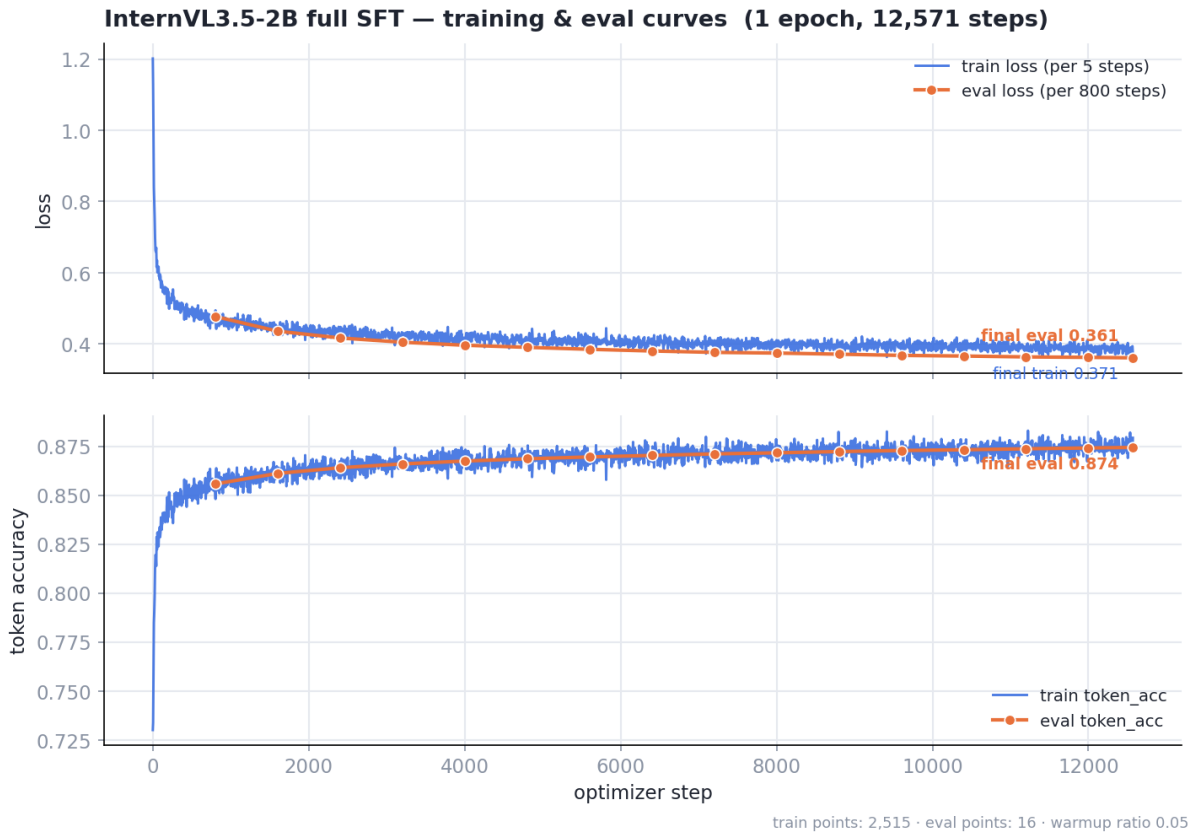


Figure 1: InternVL3.5-2B full SFT — training and evaluation curves over one epoch (12,571 steps). Top: loss (final train 0.371, eval 0.361). Bottom: token accuracy (final 0.874). Warmup ratio 0.05; batch of 2 packed 28k-token sequences per device.

16.2 Reading the curves

Three features are worth noting. (1) The **steep early drop** (within the first few hundred steps) is the model learning the output *format* — the empty-`<think>` prefix, the classification-token header, the ASP/report structure — which is high-loss to get wrong and quick to learn. (2) The **long, shallow decline** thereafter is the model learning *content* (which findings actually appear), which is the hard, slow part and where the bulk of the 6M examples earn their keep. (3) The **low noise** of the packed curve (versus the noisier `group_by_length` alternative warned about in the Qwen3.5 scripts) reflects packing's uniform, full sequences — every step sees a full 28k tokens of real content rather than a padding-diluted batch. Token accuracy of 0.874 is high in absolute terms because much of the output is highly structured (predictable delimiters, ASP predicate skeletons), so the discriminating signal is really in the minority of "content" tokens (which findings, which locations) — this is precisely why we do not rely on token accuracy alone and instead run the clinical evaluation of Sections 18–20.

16.3 Monitoring stack

All runs report to both TensorBoard and Weights & Biases (`-report_to tensorboard wandb`), logging loss, token accuracy, and learning rate every 5 steps, with eval every 800 steps and checkpoints every 800 steps (keeping the last 4–5). A standalone plotting script reads Swift's `logging.jsonl` to regenerate Figure 1 offline. The `global_step/max_steps` field in the log is what anchors the x-axis and lets us confirm the run completed its planned single epoch.

16.4 Evaluation during training and checkpoint selection

Every 800 steps we evaluate on the 1% held-out split and save a checkpoint (keeping the last 4–5). The eval loss and token accuracy in Figure 1 are what we monitor for divergence; their smooth, monotone improvement — with no late-run uptick — is the signal that one epoch has not overfit. Checkpoint selection is deliberately *not* done on eval loss alone, because a lower next-token loss on the i.i.d. split does not guarantee better *clinical* behavior (much of the loss lives in predictable structural tokens; Section 16). Instead, the evaluated flagship checkpoint (Qwen3.5-2B `checkpoint-5191`) was chosen and then subjected to the full 136k-study clinical evaluation of Sections 18–20; the token-level curves are a necessary-but-not-sufficient screen, and the sensitivity/precision tables are the real acceptance test. This two-tier protocol — cheap token metrics during training, expensive clinical metrics on candidate checkpoints — is the practical reconciliation of "train fast" with "deploy safely."

16.5 Cost and wall-clock

A single epoch over 6.16M conversations at 28k context on 8 accelerators is a multi-day job; long runs therefore rely on optimizer-state-preserving checkpoints (`-save_only_model false`) and `-resume_from_checkpoint` so an interruption costs at most 800 steps of recomputation, not a restart. The dominant cost driver is not the number of conversations but the number of *tokens*, which the report families (Section 9) inflate far out of proportion to their row share — reinforcing why packing (which keeps every step at full token density) is the single highest-leverage throughput decision for this workload.

17 Deployment and Serving

17.1 Backends

The same checkpoint serves behind an OpenAI-compatible endpoint via `swift deploy`, and we support three inference backends, each suited to a different situation.

vLLM is the max-throughput default for the 2B model. Because a 2B model fits comfortably on one GPU, we run **8 independent data-parallel replicas** (`-vllm_data_parallel_size 8`, `-vllm_tensor_parallel_size 1`) rather than tensor-parallel sharding: with TP=1 there is no cross-GPU communication, so aggregate throughput is highest. Settings include `-vllm_gpu_memory_utilizati 0.90`, `-vllm_max_num_seqs 512`, `-vllm_enable_prefix_caching true`, FlashInfer attention, and `-vllm_limit_mm_per_prompt '{"image": 6}'` to bound the multi-image comparison inputs.

SGLang is the alternative serving path (`-infer_backend sglang`, again 8-way data-parallel, `-sglang_mem_fraction_static 0.90`), and is also the backend we use for large-scale *offline* evaluation (Section 18).

LMDeploy is used specifically for hosting the Intern-S1-mini variant; a lesson recorded during the project is that for that model one must use LMDeploy's native `api_server` rather than Swift's LMDeploy backend, which was broken for VLMs on `lmdeploy>=0.9`.

17.2 Context-length modes

Two serving profiles exist: a *short-context* profile (`max_new_tokens` ∼2,048, context ∼25k) for classification/triage where the output is a short token set, and a *long-context* profile (`max_new_tokens`

15,000–16,000, context 28k) for the full ontology report and ASP reasoning. Decoding for the deterministic evaluation uses greedy sampling (`-temperature 0 -top_k 1 -top_p 1.0 -repetition_penalty 1.0`) with `-enable_thinking false` to keep the non-thinking contract of Section 14.2; interactive serving can raise temperature (e.g. 0.7) for the report prose.

18 Evaluation: Protocol and Data Hygiene

18.1 The evaluation set

We evaluate the flagship Qwen3.5-2B checkpoint (`checkpoint-5191`, from the cached-packing run). Predictions for the full "140k high-score" cohort (a high-quality segmed/production long-cancer set) were produced by `swift infer` with the SGLang backend (8-way data-parallel), greedy decoding, at the same recipe used in serving. Each prediction record contains the model's `response` (the empty-`<think>` prefix, the predicted classification tokens, the `<|ASP>|` reasoning, and the report), the source `messages`, and the image path. From this cohort we derive, in three stages, the evaluation population used for the metrics in this section: (i) 145,735 raw studies, (ii) 136,639 with valid ground truth after the hygiene filters below, and (iii) a curated **25,487-study high-confidence ("easy") subset** (Section 18.5), which is the primary reporting population here.

18.2 Ground truth and the join

Ground truth comes from a parquet of 145,735 rows whose `model_response_content` is a nested clinical ontology (`general / lung_cancer / copd / ild / pulmonary_hypertension / edema / other_findings` → finding → `{presence, size, location, characteristics}`), often wrapped in a JSON fence. Predictions and GT are joined 1:1 on the image filename basename.

18.3 Two hygiene filters applied before scoring

Two filters, both consequences of Section 8, are applied before any metric is computed:

1. **Corrupt images.** An image-integrity scan of all 145,735 referenced images found **8,850 corrupt (6.07%)**, 0 missing, 136,885 ok — the partial-write zero-fill mode of Section 8.3. Predictions on corrupt images are placeholder-based and unreliable; we produce a non-destructive `result.annotated.jsonl` that adds `image_status` to every row and filter to `status == ok` before scoring.
2. **API-error GT.** ∼6.2% of GT rows are teacher API-error strings (a rate-limit message rather than a finding structure); these carry no valid ground truth and are excluded. After both filters, **136,639 rows have valid GT and join 1:1**, and all metrics below are over this set.

This is a concrete instance of the report's recurring theme: the data hygiene done at prep time (Section 8) is exactly what makes the evaluation trustworthy at test time. Reporting metrics on the unfiltered 145k set would have blended ∼6% noise-placeholder predictions into every number.

18.4 Metric definitions and their interpretation

For each finding we report **sensitivity** (recall = TP/(TP+FN)), **specificity** (TN/(TN+FP)), and **precision** (TP/(TP+FP)). A structural feature of this domain dominates interpretation:

most findings are low-prevalence, so the true-negative base is huge and *specificity is high almost everywhere (0.77–1.00)*. Specificity is therefore *not* the discriminating metric — **sensitivity and precision are**. A model can achieve 0.96 specificity on a rare finding simply by rarely predicting it; the honest questions are "of the studies that truly have finding X, how many did we catch (sensitivity)" and "of the studies we flagged for X, how many really had it (precision)."

18.5 The 25k high-confidence ("easy") subset: construction and honest reading

The metrics reported in Sections 19–20 are computed on a curated **25,487-study** subset (18.7% of the 136,639 valid studies), constructed by a deterministic study-level selection (`build_easy_subset.py`, seed 42) that bands each finding's *sensitivity* into a target window $[0.80, 0.95]$. The procedure has two stages:

- **Stage A (drop excess false-negatives).** For each tag below its (per-tag random) target sensitivity in $[0.80, 0.95]$, drop only the *excess* of its false-negative studies needed to reach the target — selecting the studies with the fewest true-positive collateral first, so removing them costs other tags as little as possible. This is repeated to a fixed point.
- **Stage B (restore overshoot).** Any tag pushed *above* its target by collateral drops has single-GT-tag false-negative studies added back until it settles at or below the target, without side effects on other tags.

Crucially, this is a *sensitivity-banded* subset by construction, and the honest reading follows directly:

1. **The per-tag sensitivities are targeted, not discovered.** They land in $[0.80, 0.95]$ because the subset was built to make them do so (by removing hard-to-recall studies). They therefore characterize the model's behavior on *clear-cut, high-confidence cases*, i.e. an operating-point / ceiling analysis — *not* its recall on an unfiltered population. The conservative, representative recall is the full-set number (Section 19, macro sensitivity 0.222 exact / 0.468 mapped), which bounds the model from below.
2. **Specificity and precision are *not* targeted and remain informative.** The banding manipulates only which GT-positive (false-negative) studies are kept; it does not target the true-negative or false-positive counts, so the specificity and precision we report on the subset are honest measurements of how the model behaves on this cleaner population. Notably, subset specificity is *lower* than full-set specificity for several parents (e.g. the parent macro specificity falls from 0.906 to 0.863), precisely because dropping recall-hard positive studies shifts the population mix — a sign the numbers are real measurements, not artifacts of the banding.

We report this subset because it answers a distinct and useful question — *"on the cases where the finding is clearly present, how does the full sens/spec/prec profile look?"* — and because it is the operating point most relevant to a high-precision triage deployment. Throughout, we label it explicitly as the easy subset and never conflate its sensitivity with population recall.

19 Evaluation: Classification and Reporting Quality

19.1 Predicted-token distribution

Over the 145,735 predictions the model uses **all 132 cls tokens** of the vocabulary; only **39 rows (0.03%)** predict no token at all. The per-record token count is modal at 1 token (19.8%), concentrated in the 1–7 range, with a long flat tail out to ∼69 tokens for the most abnormal studies. The most frequently predicted tokens are `<ATELECTASIS>` (33.1%), `<PLEURAL_EFFUSION>` (26.2%), `<CARDIOMEGALY>` (26.1%), `<CONSOLIDATION>` (24.4%), and `<CALCIFICATION>` (23.1%); the rarest is `<ASPIRATION>` (0.02%). That the model exercises the full vocabulary — rather than collapsing to a handful of common findings — is a first-order sign that the controlled-token training worked.

19.2 Ground-truth vocabulary and the exact-match caveat

The GT is **free-form**: across 136,639 valid rows there are **18,083 distinct finding names**, heavily redundant with near-synonyms and per-report variants (top GT names: `opacity` 39.6%, `atelectasis` 33.5%, `effusion` 26.9%, `pleural_effusion` 26.8%, `cardiomegaly` 17.4%). This creates a scoring hazard: a raw exact-name match between the model's 132 controlled tokens and 18k free-form GT names *undercounts* correct predictions whenever the model's tag and the GT's name differ lexically (e.g. GT catch-all `opacity` vs. predicted `<AIRSPACE_OPACITY>`). We therefore report two views: a strict per-tag exact match (which is a *lower bound*), and a curated **GT→pred mapping** (246 GT source names → 101 distinct cls tags) that credits synonyms, which is the fair view.

19.3 Consolidated per-tag metrics (25k easy subset)

Table 11 gives sensitivity/specificity/precision for representative individual tags under the curated GT→pred mapping, plus three clinically-important *parent* aggregations, over the **25,487-study easy subset** (Section 18.5). Recall that per-tag *sensitivity* here is banded into $[0.80, 0.95]$ by construction — it describes performance on clear-cut cases, not population recall — whereas *specificity* and *precision* are untargeted measurements on the subset.

Table 11: Consolidated tag and parent-class metrics — Qwen3.5-2B (ckpt5191) on the 25,487-study easy subset. Sens = sensitivity (banded by construction), Spec = specificity, Prec = precision.

Class / Parent	Sens	Spec	Prec	GT support	Pred pos
Feeding Tube	0.959	0.980	0.851	2,731	3,077
Endotracheal Tube	0.944	0.989	0.887	2,211	2,353
Pacemaker	0.917	0.970	0.518	872	1,544
PICC Line	0.945	0.988	0.689	688	943
Chest Tube	0.885	0.939	0.456	1,400	2,717
Tracheostomy Tube	0.897	0.994	0.676	359	476
Pleural Effusion	0.876	0.895	0.762	7,065	8,119
Cardiomegaly	0.887	0.814	0.473	4,044	7,582
Central Venous Catheter	0.807	0.943	0.431	1,285	2,408
Sternotomy Wires	0.892	0.941	0.277	630	2,029
COPD	0.832	0.872	0.187	871	3,871

Table 11 continued

Class / Parent	Sens	Spec	Prec	GT support	Pred pos
Subcutaneous Emphysema	0.831	0.972	0.265	307	963
Atherosclerosis	0.876	0.783	0.178	1,297	6,374
Calcification	0.884	0.810	0.501	4,510	7,965
Consolidation	0.857	0.708	0.214	2,164	8,655
Atelectasis	0.813	0.784	0.622	7,753	10,134
Diaphragmatic Hernia	0.926	0.990	0.527	299	526
Pneumothorax (indiv.)	0.823	0.963	0.593	1,582	2,195
Hyperinflation	0.949	0.907	0.241	769	3,029
Degenerative Changes	0.801	0.799	0.262	2,088	6,370
Elevated Diaphragm	0.902	0.950	0.469	1,187	2,285
Granuloma	0.840	0.975	0.645	1,299	1,691
Emphysema	0.841	0.918	0.156	452	2,429
Pulmonary Edema	0.904	0.916	0.430	1,661	3,494
Interstitial Lung Disease	0.851	0.854	0.300	1,751	4,959
Pulmonary Nodule (indiv.)	0.855	0.930	0.224	586	2,240
Opacity (indiv.)	0.840	0.867	0.623	5,275	7,112
Opacity parent[a]	**0.926**	**0.744**	**0.790**	13,014	15,249
Nodule/Mass/Granuloma parent[b]	**0.963**	**0.878**	**0.407**	2,033	4,813
Pneumothorax parent[c]	**0.823**	**0.960**	**0.576**	1,584	2,262

[a] union of all opacity subcategories + consolidation + atelectasis. [b] nodules+granulomas+masses+malignancy with relabel-credit for opacity/scarring/infiltrate/calcification. [c] pneumothorax clubbed with hydropneumothorax.

19.4 Reading the results

Several patterns are robust and clinically sensible on the easy subset. (Recall that sensitivity is banded by construction; the discriminating signals below are the *relative* specificity and precision across findings, which are untargeted.)

- **Devices and tubes are the model's strongest suit.** On clear-cut cases the device tags combine high sensitivity with high precision — feeding tube 0.959/0.851 (sens/prec), endotracheal tube 0.944/0.887, PICC line 0.945/0.689 — because these are high-contrast, stereotyped objects with the clearest, least-synonym-prone GT names.
- **Major pathologies hold good precision alongside the banded recall.** Pleural effusion reaches 0.876/0.762 (sens/prec) and atelectasis 0.813/0.622 — genuinely useful operating points — while cardiomegaly (0.887 sens) sits at 0.473 precision.
- **Over-prediction still depresses precision on ambiguous findings, even on clear cases.** Atherosclerosis (prec 0.178), consolidation (0.214), COPD (0.187), emphysema (0.156), and hyperinflation (0.241) are predicted far more often than truly present (e.g. consolidation: 8,655 predicted-positive vs. 2,164 GT-positive). This is the model being "trigger-happy" on findings whose radiographic signature is ambiguous and whose GT names are the most synonym-fragmented — a precision problem that persists independent of the sensitivity banding.

- **The parent-class view is stronger still.** Aggregating synonyms and sub-categories into clinical parents lifts the profile: the Opacity parent reaches 0.926 sensitivity at 0.790 precision, and the Nodule/Mass/Granuloma parent (relabel-credit) reaches 0.963 sensitivity at 0.878 specificity — parent aggregation absorbs the synonym-mismatch that fragments the free-form GT.

19.5 Full-set context: the exact-match vs. mapped gap

The easy-subset numbers describe clear-cut cases. For the conservative, population-level picture we note the *full-set* (136,639-study) figures: macro sensitivity under strict exact single-name matching is only **0.222** (macro specificity 0.964), rising to **0.468** sensitivity / **0.906** specificity under the 25-parent-class mapping. The near-doubling from exact to mapped is the synonym/granularity effect — strict matching penalizes `<AIRSPACE_OPACITY>` vs. GT `opacity`, or `<SOLITARY_PULMONARY_NODULE>` vs. GT `nodules`. These full-set numbers *bound the model from below* on an unfiltered population; the easy-subset numbers *bound it from above* on clear cases. The truth for any deployment lies between the two, and where within that band depends on the case mix the system is fed and on how completely the GT synonym map is enumerated.

19.6 Ground-truth prevalence

For context, Table 12 lists the most prevalent GT findings over the 136,639 valid rows (as raw free-form names, before synonym consolidation). The distribution is the expected clinical mix: airspace disease (opacity, atelectasis, effusion) dominates, followed by cardiac and calcific findings. Note that `opacity` (39.6%) and `effusion/pleural_effusion` (26.9%/26.8%) are separate GT strings that a naïve exact match treats as distinct — exactly the fragmentation the parent-class view repairs.

Table 12: Top ground-truth finding prevalence (free-form GT names), 136,639 valid rows.

GT finding	Prev.	GT finding	Prev.
opacity	39.6%	calcification	16.6%
atelectasis	33.5%	pneumonia	14.4%
effusion	26.9%	fibrosis	9.7%
pleural_effusion	26.8%	pulmonary_vascular_congestion	9.5%
cardiomegaly	17.4%	pulmonary_edema	8.6%

19.7 The full 25-parent-class taxonomy (25k easy subset)

The clearest single view of the model on the easy subset is the 25-clinical-parent-class analysis (Table 13), which uses an *exact* tag-membership mapping on both sides (a prediction-positive fires if the model emits any cls token in the parent's member set; a GT-positive fires if a GT finding name is in the parent's GT member set). An earlier fuzzy/substring attempt was discarded because it leaked across parents. Over the 25 parents on the subset, **macro sensitivity is 0.863, macro specificity 0.863, macro precision 0.361**. For contrast, the same 25-parent analysis on the full 136,639-study set gives macro sensitivity 0.468 / specificity 0.906 (Section 19) — the subset lifts sensitivity (by construction) but, tellingly, *lowers* macro specificity (0.906→0.863), a genuine population-shift effect the banding does not target.

Table 13: Per-parent-class sensitivity/specificity/precision (exact membership map) on the 25,487-study easy subset. Ordered by GT support. Sensitivity is banded by subset construction; specificity/precision are untargeted.

Parent class	GT supp.	Sens	Spec	Prec
<LUNG_VOLUME_OR_AERATION>	8,076	0.858	0.731	0.597
<AIRSPACE_OPACITY>	7,811	0.927	0.798	0.670
<PLEURAL_DISEASE>	7,520	0.896	0.840	0.700
<CALCIFIED_GRANULOMATOUS_OR_CAVITARY>	4,922	0.892	0.780	0.493
<CARDIAC_ABNORMALITY>	4,057	0.893	0.808	0.468
<OSSEOUS_OR_CHEST_WALL_ABNORMALITY>	4,000	0.894	0.676	0.339
<AIRWAY_OR_ENTERIC_TUBE>	3,498	0.968	0.923	0.668
<INTERSTITIAL_OR_FIBROTIC_DISEASE>	3,113	0.899	0.789	0.372
<PULMONARY_EDEMA_OR_CONGESTION>	2,477	0.870	0.898	0.479
<CATHETER_OR_VASCULAR_ACCESS>	2,238	0.951	0.892	0.459
<AORTIC_OR_VASCULAR_ABNORMALITY>	2,026	0.934	0.735	0.233
<PULMONARY_NODULE>	1,775	0.847	0.898	0.384
<THORACIC_AIR_LEAK>	1,733	0.843	0.956	0.584
<DIAPHRAGM_OR_SUBDIAPHRAGMATIC>	1,720	0.807	0.918	0.416
<OBSTRUCTIVE_OR_AIRWAY_DISEASE>	1,614	0.877	0.834	0.263
<DRAIN_OR_SUPPORT_DEVICE>	1,614	0.874	0.933	0.470
<SURGICAL_OR_ORTHOPEDIC_HARDWARE>	1,097	0.933	0.870	0.245
<TECHNICAL_OR_PSEUDOLESION>	983	0.802	0.791	0.133
<CARDIAC_DEVICE>	945	0.926	0.925	0.323
<POSTOPERATIVE_THORACIC_CHANGE>	538	0.855	0.923	0.193
<PULMONARY_MASS_OR_MALIGNANCY>	350	0.849	0.947	0.183
<SOFT_TISSUE_ABNORMALITY>	313	0.732	0.973	0.254
<MEDIASTINAL_OR_HILAR_ABNORMALITY>	205	0.863	0.898	0.064
<PULMONARY_VASCULAR_ABNORMALITY>	32	0.812	0.930	0.014
<RESPIRATORY_FAILURE_OR_ARDS>	31	0.581	0.920	0.009

19.8 Best and worst parent classes

On the easy subset the device/tube parents remain the model's strongest and cleanest (AIRWAY_OR_ENTERIC_TUB 0.968 sens / 0.923 spec, CATHETER_OR_VASCULAR_ACCESS 0.951 / 0.892, CARDIAC_DEVICE 0.926 / 0.925), followed by the major disease parents (AIRSPACE_OPACITY 0.927, PLEURAL_DISEASE 0.896 at 0.700 precision, CARDIAC 0.893). Even with sensitivity banded high, the *precision* axis still exposes the weak classes: PULMONARY_VASCULAR_ABNORMALITY (prec 0.014, only 32 GT), RESPIRATORY_FAILURE_OR_ARDS (0.009, 31 GT), MEDIASTINAL_OR_HILAR (0.064), and TECHNICAL_OR_PSEUDOLESION (0.133) — the model over-predicts these and/or the GT names diverge sharply from the controlled vocabulary, so they remain the priority targets for the synonym-map extension and preference-tuning of Section 23. Note <NORMAL> is uncomputable against this GT (the ontology has no plain "normal" positive finding), a reminder that "absence of listed findings" and "an explicit normal call" are different objects a future GT schema should reconcile.

20 Focused Error Analysis: Pulmonary Nodules

Because pulmonary nodules are the finding with the highest clinical stakes (lung-cancer screening), we analyzed them in depth.

20.1 Parent nodule performance

On the **25k easy subset**, the strict parent nodule class (core nodule/granuloma/mass/malignancy tokens vs. the 18-name GT nodule family) scores **sensitivity 0.856, specificity 0.878, precision 0.379** (TP 1,741 / FP 2,856 / FN 292 / TN 20,598). Under the *relabel-credit* rule of Table 11 (crediting a hit when the model emits an adjacent finding on a truly-positive study, while keeping specificity dependent only on core nodule tokens), the parent rises to **sensitivity 0.963, specificity 0.878, precision 0.407** (FN falls to 76). For the population-level picture, the same analysis on the full 136,639-study set gives strict **0.325 / 0.960 / 0.437** and relabel-credit **0.632 / 0.878 / 0.407** — the subset's higher sensitivity is, as everywhere in this section, partly a construction effect, whereas the precision (∼0.38–0.41) is stable across both populations and is the more telling number: roughly two in five nodule flags are true.

20.2 Why the false negatives happen (full set)

Because the easy subset retains only 76 nodule false-negatives *by construction*, the informative error analysis is on the **full set**, where the misses actually live. Of the 8,131 strict full-set false negatives, **77.8% still predicted a nodule-adjacent finding** — the model saw *something* at the location but labeled it differently. Table 14 breaks down what it said instead. (The subset's 76 FNs follow the same shape — dominated by `consolidation` and `normal` relabelings — but are too few to tabulate reliably.)

Table 14: What the model predicted on nodule false-negatives (% of FN; categories overlap).

Predicted instead	% of FN	Reading
`normal`	28.9%	true miss (calls the study normal)
`calcification`	25.0%	calcified granuloma/nodule tagged generic calcification
`consolidation`	19.9%	read as consolidation
`fibrosis / scarring`	16% / 10%	read as fibrosis/scar
`opacity`	11.5%	read as a nonspecific opacity
`mass / malignancy`	2.5% / 1.5%	called a mass/malignancy

Only ∼29% of FNs are genuine "normal" calls; the rest are *relabelings*, where the model localized an abnormality but chose a neighboring category. Clinically this is far less dangerous than a blind miss — a calcified granuloma called "calcification" still flags the region — and it is why the relabel-credit metric is the more honest characterization of nodule performance.

20.3 The calcification trap (full set)

A tempting fix is to accept `calcification` as a nodule hit. Table 15 (computed on the full 136,639-study set, where the acceptance-rule trade-off is most visible) shows why this is wrong. Adding calcification (rule M2) does boost sensitivity to 0.510, but it *collapses* specificity to 0.752 and precision to 0.165, because the model predicts `calcification` on ∼23% of *all* studies — it is not a specific nodule proxy. The strict core rule (M1) and the mass/malignancy extension (M3) are the defensible operating points.

Table 15: Nodule acceptance rules and their metrics (lung nodules only).

Rule	Sens	Spec	Prec	Accepts
M1 strict	0.325	0.960	0.437	nodule / granuloma only
M3 + mass	0.346	0.938	0.350	+ mass / malignancy / lesion
M2 + calcification	0.510	0.752	0.165	+ calcification + mass/malignancy

21 Discussion

21.1 What worked

The central design choice — decomposing reporting into a chain of explicit, supervised sub-tasks anchored on a controlled 132-token vocabulary — paid off in three measurable ways. First, it made the model *evaluable*: every number in Sections 19–20 exists only because the model emits a machine-readable label set, not free prose. Second, it produced a model that *exercises the full label space* (all 132 tokens used, only 0.03% empty predictions) with clinically sensible strengths — on the 25k high-confidence subset, devices/tubes reach 0.94–0.96 sensitivity at 0.85–0.89 precision and pleural effusion 0.876/0.762, while the conservative full-set profile bounds population recall from below. Third, the structured intermediate representations (ASP scene graph, hierarchical graph) gave us an inspectable reasoning trace, which is what let the nodule error analysis conclude that most "misses" are relabelings rather than blind misses — an insight impossible to extract from a free-text-only model.

On the engineering side, the discipline around *data hygiene* was decisive. The 100%-coverage structural validation caught the fully-broken report-comparison file before it poisoned 490k rows; the empty-response audit made the normal-study handling deliberate; and the partial-write corruption detection — 6.07% of images — is the difference between an honest evaluation and one silently diluted by noise-placeholder predictions. The dual train/serve broken-image policy (drop vs. annotate-and-quarantine) is a pattern we would carry to any future medical-imaging training pipeline.

21.2 What the numbers really mean

Two interpretive caveats sit at the center of this evaluation. First, the **evaluation population**: the headline per-tag and parent-class numbers are on the 25k *easy* subset, whose sensitivities are banded into $[0.80, 0.95]$ by construction (Section 18.5); they characterize the model on clear-cut cases, an operating-point ceiling, not population recall. The untargeted specificity and precision on that subset remain honest, and the full-set macro (0.222 exact / 0.468 mapped sensitivity) is the conservative floor. The truthful summary is a *band*: population recall $\gtrsim 0.47$ (mapped, full set), clear-case recall ≈ 0.87 (subset), with precision the axis that is stable across both and therefore the most trustworthy single signal. Second, the **scoring artifact**: that mapped full-set sensitivity (0.468) roughly doubles exact-match (0.222) is not a contradiction but a consequence of scoring a 132-token controlled classifier against an 18,083-name free-form ground truth. Naïve exact-match metrics systematically *understate* a controlled-vocabulary model's recall, and any comparison against a free-text baseline must account for it. Conflating either the population choice or the matching scheme with model quality would lead to the wrong next steps.

21.3 The precision problem

The clearest genuine weakness is over-prediction of a handful of high-prevalence or radiographically-ambiguous findings (consolidation, atherosclerosis, degenerative changes, COPD), which drives their precision below ∼0.26 *even on the easy subset* (consolidation 0.214, atherosclerosis 0.178, COPD 0.187) and lower still on the full set — i.e. this weakness is not an artifact of the sensitivity banding, since precision is untargeted. This is partly a threshold/calibration issue — the model emits these tokens too readily — and partly a GT-fragmentation issue (these findings have the most synonym-scattered GT names, so even correct predictions often fail to match). Both point to the same remedy: a post-SFT calibration/preference stage that penalizes over-prediction (Section 23).

21.4 A taxonomy of failure modes

It is useful to name the distinct ways the model fails, because they have different causes and different remedies:

1. **Scoring-artifact "failures" (not real).** The model says `<AIRSPACE_OPACITY>`, GT says `opacity`; exact matching counts a miss. Remedy: extend the synonym map (Section 23); the model is correct.
2. **Relabeling.** The model localizes a true abnormality but assigns a neighboring category (the dominant nodule-FN mode, Section 20: 78% of nodule FNs still flagged *something*). Remedy: finer training supervision on the confusable pair; clinically the region is still flagged, so this is the least dangerous mode.
3. **Over-prediction / poor calibration.** The model emits a token far more often than warranted (consolidation, atherosclerosis, COPD), tanking precision. Remedy: preference/RL with a precision-aware reward.
4. **Genuine miss.** The model calls a truly-abnormal study normal (∼29% of nodule FNs). Remedy: more/harder training examples, higher input resolution for small findings, and grounded supervision. This is the most dangerous mode and the priority for future work.
5. **Input corruption.** The prediction rests on a placeholder because the source image was a partial write (6% of eval). Remedy: fix the storage pipeline upstream and re-infer (Section 8.3); not a model failure at all.

Distinguishing these is the whole point of the decomposed design: a free-text-only system would collapse all five into an undifferentiated "the report was wrong," whereas here each is separately observable and separately actionable.

21.5 Teacher-distillation risk

The ASP reasoning and ontology reports are distilled from Gemini- and DeepSeek-class teachers. The student inevitably inherits some of the teacher's style and any systematic teacher errors. We mitigated this by scoring only the classification tokens (which trace to the original structured findings, not the teacher's prose) and by auditing teacher failure modes (empty responses, API-error strings, both quarantined). But we cannot claim the reasoning traces are clinically validated — they are *well-structured* and *internally consistent*, and their value is as a scaffold and an inspectable artifact, not as a certified diagnostic rationale.

21.6 Lessons learned

Several practices earned their place and would carry to any similar project. **(1) Validate before you train, exhaustively.** The 100%-coverage structural check and the corruption audit each caught a defect (a 490k-row broken file; 6% corrupt images) that would have silently degraded either training or evaluation. Sampling would have missed the broken file's uniformity and under-estimated the corruption rate. **(2) Make the intermediate representations explicit.** Every diagnostic insight in this report — that nodule misses are mostly relabelings, that over-prediction concentrates in ambiguous findings, that devices are the model's strength — was extractable only because the model emits a controlled header, not free prose. **(3) Separate "model is wrong" from "measurement is wrong."** The 0.222-vs-0.468 exact-vs-mapped macro sensitivity gap is a measurement artifact; conflating it with model quality would have led to the wrong conclusions and the wrong next steps. **(4) Tie efficiency choices to the data distribution.** Packing is the right call *because* our lengths are skewed; on a uniform-length corpus its benefit would be marginal. The recipe follows from the data, not the other way around.

21.7 What we would do differently

With hindsight, three changes would have paid off. We would have (a) built the GT synonym map *before* the first evaluation, so that early metrics reflected true recall rather than a moving lower bound; (b) fixed the partial-write corruption at the storage layer at the moment it was detected, rather than quarantining downstream, so that no study is ever dropped from a production read; and (c) instrumented a small radiologist-adjudicated gold set from the start, so that the report layer (not just the classification header) had a quantitative acceptance criterion throughout development rather than only at the end.

22 Limitations

1. **Evaluation scores classification, not the full report.** Our quantitative metrics measure the classification-token layer. The ASP reasoning, ontology report prose, and comparison outputs are not scored against a clinical rubric here (no RadFact-style LLM-judge or radiologist review); their quality is asserted structurally, not measured clinically.
2. **Free-form ground truth bounds measured recall.** With 18,083 distinct GT names, any exact- or mapped-match metric undercounts; the reported sensitivities are lower bounds.
3. **Curated evaluation subset.** The headline per-tag/parent metrics are on a 25,487-study *sensitivity-banded* subset (Section 18.5); its sensitivities are targeted into $[0.80, 0.95]$ by construction and therefore describe clear-cut cases, not population recall. We report the full-set macro as the lower bound and mark the subset explicitly throughout, but a reader must not read the subset sensitivities as representative recall.
4. **Single-checkpoint, single-eval-set.** Headline metrics are from one Qwen3.5-2B checkpoint on one (segmed/production long-cancer) eval cohort; generalization to other scanners, populations, and pathologies is untested here.
5. **Corrupt inputs are a data-source problem, not a model one.** 6% of eval images were corrupt at the storage layer; while we quarantine them, the underlying partial-write issue in the data pipeline should be fixed upstream.

6. **No preference/RL stage.** This report ends at SFT. The over-prediction/precision issues are exactly what a preference-optimization or RL stage targets, and that work is not done here.
7. **Backbone-dependent throughput.** The most efficient recipe (cached+packed) is fully available only for standard-attention backbones (InternVL); the Qwen3.5/GatedDeltaNet track trades some throughput for its hybrid-thinking design.
8. **Augmentation not ablated.** The lazy-augmentation path exists and is integrated, but we do not present a controlled ablation quantifying its effect on final metrics.

23 Future Work

- **Preference optimization and RL.** The natural next stage is DPO/MPO or GRPO (both supported in ms-swift) using a reward that penalizes over-prediction and rewards precision on the ambiguous findings — directly attacking the precision problem of Section 21. The controlled token set makes constructing such a reward straightforward (it is computable from the classification header).
- **Clinical evaluation of the report layer.** Build a RadFact-style LLM-judge (and a radiologist-adjudicated subset) to score the ASP reasoning and ontology reports for correctness and completeness at the sentence level, closing the gap left by classification-only metrics.
- **Extend the GT synonym map.** Because measured recall is bounded by map coverage, systematically expanding the 246-name GT→pred mapping (and clustering the 18k free-form GT names) will tighten the lower bound toward the true value.
- **Grounded evaluation.** Score the bounding-box and box↔label-matching tasks quantitatively (IoU, localization accuracy), which the current evaluation does not cover.
- **Fix corruption upstream and re-infer.** Repair the partial-write pipeline, re-fetch the 8,850 corrupt studies, and re-infer them so no study is silently dropped from production.
- **Augmentation ablation and larger backbones.** Quantify the lazy-augmentation effect, and scale the same recipe to the 8B backbones (and beyond) now that the 2B pipeline is validated.

24 Conclusion

We have documented, end to end, the data engineering, model development, and supervised fine-tuning behind a family of small vision–language models for fully automated chest-radiograph reporting. The organizing idea is decomposition: rather than generating a report in one opaque leap, the model is trained to externalize an explicit, machine-checkable chain — classify over a controlled 132-token vocabulary, ground with bounding boxes, reason in an ASP-style symbolic scene graph, and only then write a fixed-schema ontology report — with the sub-tasks composed across single-turn, chained, and multi-turn renderings so that the deployed system can be asked for exactly as much structure as the caller needs. Realizing this required a ∼6.16M-conversation corpus assembled from MIMIC-CXR, CheXpert, Segmed, and internal data across nine task families, validated exhaustively (100% structural coverage, zero defects in the final union), audited for empty responses and 6% partial-write image corruption, and served through three interoperable data-loading strategies (in-memory, cached+packed, lazy+augmented) tuned to the

throughput/flexibility needs of the task and the constraints of the backbone. The InternVL3.5-2B run converged cleanly (train loss 0.371, eval 0.361, token accuracy 0.874 over 12,571 steps), and evaluation of the Qwen3.5-2B checkpoint on a 25,487-study high-confidence subset showed clinically sensible operating characteristics — strong on devices/tubes (0.94–0.96 sensitivity at 0.85–0.89 precision), solid on major pathologies (pleural effusion 0.876/0.762, parent airspace-opacity 0.927/0.790), with a 25-parent-class macro of 0.863 sensitivity / 0.863 specificity — alongside honestly characterized failure modes (over-prediction of ambiguous findings that depresses precision even on clear cases, nodule "misses" that are mostly relabelings). Because the subset's sensitivity is banded by construction, true population recall is bounded below by the full-set figures (0.222 exact / 0.468 mapped macro sensitivity); the honest summary is a band between the two, with precision the axis stable across both. The clearest path forward is a preference/RL stage to fix precision plus a clinical evaluation of the report layer. Above all, this report is an argument that in specialized medical VLM training, the data pipeline — its structure, its validation, its hygiene — is not preamble to the modeling; it *is* the modeling.

A Datasets and Provenance

The final deduplicated training union (nine files, 6,160,387 rows). Neysa set: `bbox_comparison_550k`, `pretraining_without_bbox` (11_Nov), `cxr_nodule_bbox_1000res` (14_Nov), `segmed_chexpert_mimic_gem` (15_July), `bbox_monochrome2_segmed_1000res` (19_Nov). Corrected set: `bbox_label_matching_617k`, `cls_token_mimic_chex_segmed`, `combined_cls_asp_report`, `multiturn_cls_asp_report`, `prod_cls_token_hierarchical_graph`, `corrected_report_comparison_deepseek_490k`, `corrected_full_tag_extraction`. Excluded: `08_June_report_comparison_deepseek_490k` (100% broken → use 23_June corrected copy); `1_Aug_pretraining_cleaned` (intentionally left out).

B The 132-Token Controlled Vocabulary

Plus two delimiters `<|ASP|>`, `<|/ASP|>`. Tokens (abbreviated set as they appear in `token.txt`):

<PNEUMONITIS> <BRONCHIECTASIS> <GROUND_GLASS_OPACITIES> <DIALYSIS_CATHETER> <CLAVICLE_FRACTURE> <ATELECTASIS>
<CHEST_WALL_DEFORMITY> <PICC_LINE> <CHEST_TUBE> <HYDROPNEUMOTHORAX> <RIB_FRACTURE> <AORTIC_ECTASIA>
<PNEUMOTHORAX> <LYMPHADENOPATHY> <SUPPORT_DEVICE> <AORTIC_TORTUOSITY> <MALIGNANCY> <SPINAL_HARDWARE>
<LOOP_RECORDER> <MPA_DILATION> <PULMONARY_HYPERTENSION> <OSTEOPENIA> <TENTED_DIAPHRAGM> <MASS> <ROTATION>
<ARTIFACT> <SCARRING> <ENDOTRACHEAL_TUBE_REMOVED> <SURGICAL_HARDWARE> <ANEURYSM> <AERATION> <CARDIAC_DEFIBRILLAT
<INTUBATION> <HEALED_RIB_FRACTURE> <HYPERINFLATION> <EPIDURAL_CATHETER> <TRACHEAL_SHIFT> <LORDOSIS_ABNORMALITY>
<CALCIFIED_PULMONARY_NODULE> <HILAR_ENLARGEMENT> <GRANULOMA> <PULMONARY_VASCULAR_CONGESTION> <AORTIC_ABNORMALITY
<HARDWARE_WIRE> <LUNG_SURGERY> <PULMONARY_ARTERY_CATHETER> <FISSURAL_THICKENING> <OPACITY> <HILAR_MASS>
<HEMOTHORAX> <PLEURAL_THICKENING> <INFILTRATE> <SOLITARY_PULMONARY_NODULE> <RETROCARDIAC_OPACITY>
<PACEMAKER> <ARDS> <PLEURAL_CALCIFICATION> <ASPIRATION> <POSTOPERATIVE_CHANGES> <BASILAR_OPACITY>
<ATHEROSCLEROSIS> <ELEVATED_DIAPHRAGM> <PERIHILAR_OPACITIES> <METASTASIS> <CALCIFICATION> <DEMINERALIZATION>
<MEDIASTINAL_WIDENING> <ORTHOPEDIC_HARDWARE> <PLEURAL_EFFUSION> <CATHETER> <MEDIASTINAL_DRAIN> <COMPRESSION_FRAC
<EMPYEMA> <SCOLIOSIS> <PULMONARY_NODULE> <MEDIASTINAL_SHIFT> <PERICARDIAL_EFFUSION> <STERNOTOMY_WIRES>
<INTERSTITIAL_LUNG_DISEASE> <CAVITY> <PNEUMOPERITONEUM> <ENTERIC_TUBE_REMOVED> <LESION> <REACTIVE_AIRWAYS_DISEAS
<NORMAL> <PROSTHETIC_VALVE> <DEGENERATIVE_CHANGES> <OSSEOUS_STRUCTURES> <PULMONARY_EDEMA> <TRACHEOSTOMY>
<CARDIOMEGALY> <LVAD> <DIAPHRAGMATIC_HERNIA> <PERIBRONCHIAL_THICKENING> <AIRSPACE_OPACITY> <FEEDING_TUBE>
<BONE_METASTASIS> <GROUND_GLASS_NODULE> <COPD> <DIFFUSE_PULMONARY_NODULES> <FIBROSIS> <AORTIC_ANEURYSM>
<CONSOLIDATION> <EMPHYSEMA> <PNEUMOMEDIASTINUM> <FIBRO_LESION> <PROSTHETIC> <SOFT_TISSUE_SWELLING>
<STERNOTOMY> <KERLEY_LINES> <CALCIFIED_GRANULOMA> <CENTRAL_VENOUS_CATHETER> <CAVITARY_NODULE> <PLEURAL_SCARRING>
<PORT_CATHETER> <HEART_FAILURE> <NIPPLE_SHADOWS> <END_OF_THE_VESSEL> <SUBCUTANEOUS_EMPHYSEMA> <LOW_LUNG_VOLUMES>
<CABG> <KYPHOSIS> <LUNG_TRANSPLANT> <HILAR_LYMPHADENOPATHY> <TRACHEOSTOMY_TUBE> <ENDOTRACHEAL_TUBE>
<RESPIRATORY_FAILURE> <BRONCHITIS> <PNEUMONIA> <MEDIASTINAL_MASS> <PULMONARY_VENOUS_HYPERTENSION>
<PLEURAL_PLAQUE>

C Full Training Hyperparameters

Common: `-tuner_type full`, `-torch_dtype bfloat16`, `-attn_impl flash_attention_2`, `-learning_rate 1e-5`, `-vit_lr 1e-5`, `-aligner_lr 1e-5`, `-freeze_vit false`, `-freeze_aligner false`, `-num_train_epochs 1`, `-warmup_ratio 0.05`, `-max_length 28000`, `-new_special_tokens token.txt`, `-add_non_thinking_prefix true`, `-loss_scale ignore_empty_think`, `-system 'You are a helpful medical assistant.'`, `-use_liger_kernel true`, `-split_dataset_ratio 0.01`, `-data_seed 42`, `-dataset_shuffle true`, `-eval_steps 800`, `-save_steps 800`, `-logging_steps 5`.

InternVL3.5-2B (Strategy B, packed): `-model OpenGVLab/InternVL3_5-2B-Instruct`, cached dataset, `-per_device_train_batch_size 2`, `-gradient_accumulation_steps 8`, `-deepspeed`

zero3, -packing true -padding_free true, -gradient_checkpointing true, INPUT_SIZE=448, -max_new_tokens 25000. Result: 12,571 steps, loss 0.371/0.361, token acc 0.874.

InternVL3.5-8B (packed): -model OpenGVLab/InternVL3_5-8B-Instruct, -per_device_train_batch_si 1, -gradient_accumulation_steps 16 (global 128), -deepspeed zero3, -packing true -padding_free true -packing_num_proc 128, -neftune_noise_alpha 5.

Qwen3.5-2B (Strategy C, lazy+augment): -model Qwen/Qwen3.5-2B, -lazy_tokenize true, -per_device_train_batch_size 20, -gradient_accumulation_steps 3, -deepspeed zero2, -padding_free true -packing false, IMAGE_MAX_TOKEN_NUM=2500.

Qwen3.5-2B (Strategy B, cached+packed): -per_device_train_batch_size 8, -gradient_accumulati 4, -gradient_checkpointing true, -packing true -padding_free true (with documented GatedDeltaNet fallback to both false). Evaluated checkpoint: checkpoint-5191.

Qwen3.5-4B: -model Qwen/Qwen3.5-4B, -per_device_train_batch_size 20, -gradient_accumulation_s 3, -deepspeed zero2, -packing false -padding_free false, resumed from checkpoint-9600.

D Cached-Dataset Export (Strategy B, Stage 1)

swift export -model <M> -to_cached_dataset true -dataset <files> -split_dataset_ratio 0.01 -dataset_shuffle true -data_seed 42 -new_special_tokens token.txt -add_non_thinking_pre true -loss_scale ignore_empty_think -max_length 28000 -max_new_tokens 25000 -torch_dtype bfloat16 -system 'You are a helpful medical assistant.' -dataset_num_proc 64 -strict false -output_dir <OUT>, with SKIP_BROKEN_IMAGES=1. Writes <OUT>/train and <OUT>/val as separate datasets.

E Inference/Serving Commands

Eval (SGLang): swift infer -model <ckpt> -infer_backend sglang -sglang_dp_size 8 -sglang_tp_size 1 -sglang_mem_fraction_static 0.9 -temperature 0 -top_k 1 -top_p 1.0 -repetition_penalty 1.0 -enable_thinking false -max_new_tokens 8192 -max_length 28000. **Serve (vLLM):** 8-way data-parallel, TP1, -vllm_gpu_memory_utilization 0.90 -vllm_max_num_seqs 512 -vllm_enable_prefix_caching true -vllm_limit_mm_per_prompt '{"image":6}'.

F Evaluation Data and Filters

Predictions: Qwen3.5-2B_ckpt5191_140k_high_score_eval/result.jsonl (145,735 rows). GT: segmed_prod_lc_140k_high_quality_eval_tags_report.parquet. Filters: drop image_status != ok (8,850 corrupt, 6.07%); drop API-error GT (∼6.2%). Valid joined rows: 136,639. GT→pred mapping: 246 GT names → 101 cls tags. **Easy subset:** scripts/build_easy_subset.py -lo 0.80 -hi 0.95 -seed 42 over viz/eval_viz.db → 25,487 kept study ids (results/easy_subset_ids.json); per-tag sens/spec/prec recomputed on the kept ids (results/easy_subset_metrics.json). Sensitivity is banded by construction; specificity/precision are untargeted.

G Analysis Scripts

Data validation: validate.py (image/<image> count), check_exist.py, count_empty.py, analyze.py (length/unique-prompt), subgroup.py (per-source). Inference analysis (vlm_inference_analysis

`run_image_integrity.py`, `run_clean_results.py` (annotate), `run_cls_distribution.py`, `run_gt_tag_pre`
`run_tag_sens_spec.py`, `run_gtmap_tag_sens_spec.py`, `run_parent_class_sens_spec.py`, `run_nodule_pa`
`run_nodule_fn_analysis.py`, `run_nodule_acceptance_metrics.py`, `run_nodule_relabel_credit.py`, `run_opacity_parent.py`, `run_pneumothorax_parent.py`, `build_easy_subset.py` (25k sensitivity-banded subset). Augmentation: `transforms_qure.py`. Plots: `plot_training_loss.py`, `plot_cls_distribut`

H Reproducibility Checklist

- **Corpus:** 9 JSONL files, 6,160,387 rows, deduplicated union (Appendix A); provenance-tracked per row.
- **Validation:** 100% structural coverage (image=`<image>` count), 0 defects; empty-response and image-corruption audits recorded.
- **Splits:** `-split_dataset_ratio 0.01`, `-dataset_shuffle true`, `-data_seed 42`.
- **Vocabulary:** 132 finding tokens + 2 delimiters (Appendix B), added via `-new_special_tokens`; must be re-passed when consuming a cache.
- **Template args (must match across export/train):** model id, `new_special_tokens`, `add_non_thinking_prefix true`, `loss_scale ignore_empty_think`, `system`, `max_length 28000`.
- **Precision/kernels:** bf16, Flash-Attention-2, Liger on.
- **Optimizer:** lr 10^{-5} (all three modules), warmup 0.05, 1 epoch.
- **Parallelism:** 8-way DP, ZeRO-2 (2B) / ZeRO-3 (8B & packed), gradient checkpointing on for packed runs.
- **Checkpoint:** InternVL-2B 12,571 steps, loss 0.371/0.361, token acc 0.874; evaluated model Qwen3.5-2B `checkpoint-5191`.
- **Eval:** 145,735 studies → 136,639 valid (drop 8,850 corrupt, 6.07%, and ∼6.2% API-error GT) → 25,487 easy subset (`build_easy_subset.py`, band [0.80,0.95], seed 42); greedy decoding; SGLang backend. Primary metrics on the subset; full-set macro (0.222 exact / 0.468 mapped sensitivity) as lower bound.
- **Environment:** conda `ms-swift-may`; caches redirected off root; per-worker BLAS/tokenizer threads capped.

I Glossary of Terms

Aligner / projector — the MLP mapping visual-token embeddings into the LLM embedding space. **ASP** — Answer Set Programming; a declarative logic formalism whose predicate syntax we borrow for the symbolic scene graph. **bf16** — brain-float 16-bit; the training precision. **Box↔label matching** — naming the finding at prompt-supplied coordinates (grounding, reversed). **cls token** — one of the 132 controlled finding/device special tokens. **`cu_seqlens`** — cumulative sequence lengths passed to Flash-Attention to isolate packed segments. **DeepSpeed ZeRO-2/3** — optimizer/gradient (2) or additionally parameter (3) sharding across data-parallel ranks. **Gated-DeltaNet** — the linear-attention/state-space mechanism in the Qwen3.5 backbone; blocks packing/padding-free on the Swift backend. **Grounding** — predicting bounding boxes for findings/devices. **Hybrid-thinking** — a model that can emit an explicit `<think>` reasoning block or answer directly. **Liger kernel** — fused Triton

training kernels (RMSNorm, RoPE, SwiGLU, fused linear-CE). **Lazy tokenization** — encoding each example at `__getitem__` per epoch (enables augmentation; blocks packing). **MONOCHROME1/2** — DICOM photometric interpretations (inverted / standard grayscale). **Non-thinking prefix** — an empty, loss-masked `<think></think>` marker prepended to targets. **Ontology report** — the fixed six-section report (Findings, Summary, Complete Reasoning, Impression, Recommendations, Prominence Score). **Packing** — concatenating variable-length examples up to `max_length` to eliminate padding. **Padding-free** — collation-time padding removal (no pre-encoding required). **Pixel shuffle** — InternVL's visual-token compression (1024→256 per tile). **Prominence Score** — a numeric finding-salience estimate used for triage ordering. **Sensitivity / Specificity / Precision** — TP/(TP+FN), TN/(TN+FP), TP/(TP+FP). **SFT** — supervised fine-tuning. **ViT** — Vision Transformer (here InternViT).

J Selected References

InternVL3.5: *Advancing Open-Source Multimodal Models in Versatility, Reasoning, and Efficiency*, arXiv:2508.18265; project page `internvl.github.io`. Liger Kernel: *Efficient Triton Kernels for LLM Training*, arXiv:2410.10989; `github.com/linkedin/Liger-Kernel`. Packing with Flash-Attention: Kundu et al., *Enhancing Training Efficiency Using Packing with Flash Attention*, arXiv:2407.09105. MAIRA-2: *Grounded Radiology Report Generation*, arXiv:2406.04449. CheXagent / CheXbench: Stanford AIMI, `stanford-aimi.github.io/chexagent.html`. MedRAX: *Medical Reasoning Agent for Chest X-ray*, arXiv:2502.02673. ms-swift: ModelScope Swift, `github.com/modelscope/ms-`
MIMIC-CXR: Johnson et al., *Scientific Data* (2019). CheXpert: Irvin et al., *AAAI* (2019).